

LAPORAN TUGAS UTS
LAB. SENSOR DAN TRANSDUSER
MONITORING KETINGGIAN AIR BERBASIS IOT
MENGGUNAKAN ESP8266 DAN BLYNK



DISUSUN OLEH :

Nama : Hema Tata Nugraha
Kelas : EK-2C
Nomor Absen : 11
NIM : 3.32.24.2.13

Dosen Pengampu : Wahyu Sulistiyo, S.T., M.Kom.

PROGRAM STUDI D3 TEKNIK ELEKTRONIKA
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI SEMARANG
TAHUN AJARAN 2025



BIODATA PRAKTIKAN



Nama : Hema Tata Nugraha

NIM : 3.32.24.2.13

No. Absen : 11

Kelas : EK-2C

PROJEK UTS SENSOR ULTRASONIK HC-SR04

I. Tujuan

1. Membuat sistem otomatis untuk memantau dan mengendalikan ketinggian air menggunakan sensor ultrasonik HC-SR04.
2. Mengontrol pompa air secara otomatis dengan modul relay berbasis ESP8266.
3. Menampilkan data level air dan status pompa pada LCD dan aplikasi Blynk.
4. Menerapkan dua mode kerja, yaitu otomatis dan manual melalui aplikasi Blynk.

II. DASAR TEORI

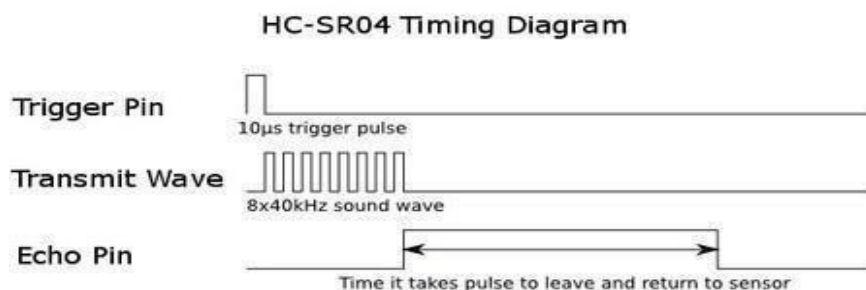
HC-SR04 adalah sensor non-kontak pengukur jarak yang menggunakan ultrasonik. Sensor ini hanya memerlukan 2 pin I/O untuk berkomunikasi dengan mikrokontroler, yaitu TRIGGER dan ECHO. Untuk mengaktifkannya, mikrokontroler mengirimkan pulsa positif melalui pin TRIGGER minimal 10 μ s, selanjutnya sensor akan mengirimkan 8 sinyal berfrekuensi 40kHz dan mendeteksi adakah sinyal yang kembali ke sensor karena dipantulkan oleh suatu objek. Jika terdapat sinyal yang kembali ke sensor maka akan terbaca oleh receiver dari sensor. Rentang waktu dari sinyal yang dikirim hingga diterima akan berbanding lurus dengan jarak dari objek yang memantulkan sinyal tersebut.



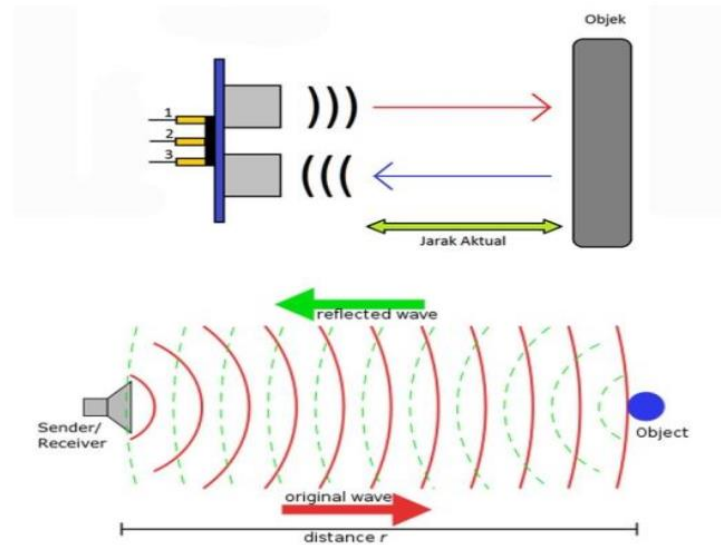
Gambar 1. Sensor Ultrasonik HC-SR04

Fungsi Pin-pin HC-SR04

1. VCC = 5V
2. Trig = Trigger/Penyulut. Pin ini yang digunakan untuk membangkitkan sinyal ultrasonik.
3. Echo = Receive/Indikator. Pin ini yang digunakan untuk mendeteksi sinyal pantulan ultrasonik.
4. GND = Ground



Gambar 2. Diagram waktu cara kerja sensor HC-SR04



Gambar 3. Prinsip kerja sensor ultrasonik

Secara detail, cara kerja sensor ultrasonik adalah sebagai berikut:

- Sinyal dipancarkan oleh pemancar ultrasonik dengan frekuensi tertentu dan dengan durasi waktu tertentu. Sinyal tersebut berfrekuensi diatas 20kHz. Untuk mengukur jarak benda (sensor jarak), frekuensi yang umum digunakan adalah 40kHz.
- Sinyal yang dipancarkan akan merambat sebagai gelombang bunyi dengan kecepatan sekitar 340 m/s. Ketika menumbuk suatu benda, maka sinyal tersebut akan dipantulkan oleh benda tersebut.
- Setelah gelombang pantulan sampai di alat penerima, maka sinyal tersebut akan diproses untuk menghitung jarak benda tersebut. Jarak benda dihitung berdasarkan rumus

$$S = \frac{340 \cdot t}{2}$$

$$S = 340 \cdot t/2$$

Dimana : S = Jarak antara sensor dengan objek (m)

t = Waktu tempuh gelombang ultrasonik dari transmitter ke receiver

Kecepatan suara di udara pada suhu 25 °C ≈ 343 m/s
Ubah ke cm/μs:

$$343 \text{ m/s} = 34300 \text{ cm/s}$$

$$1 \text{ s} = 1,000,000 \text{ } \mu\text{s}$$

$$v = \frac{34300}{1,000,000} = 0.0343 \text{ cm}/\mu\text{s}$$

$$\text{jarak (cm)} = \frac{v \times t}{2}$$

$$\text{jarak (cm)} = \frac{0.0343 \times t}{2} = t \times 0.01715$$

$$t = \frac{\text{jarak}}{0.01715} = \text{jarak (cm)} \times 58.3$$

$$\boxed{\text{jarak (cm)} = \frac{t(\mu\text{s})}{58.3}}$$



Spesifikasi Sensor HC-SR04 :

Power Supply	: +5V DC
Arus daya	: 15 mA
Sudut efektif	: $<15^{\circ}$
Pembacaan jarak	: 2 cm – 400 cm
Pengukuran sudut	: 30°



III. ALAT DAN BAHAN

No	Nama Komponen	Jumlah	Fungsi
1	NodeMCU ESP8266	1	Mikrokontroler utama dan koneksi WiFi ke Blynk
2	Sensor Ultrasonik HC-SR04	1	Mendeteksi jarak permukaan air
3	Modul Relay 1 Channel (5V)	1	Mengontrol pompa air otomatis
4	Resistor 20KΩ	1	Untuk menyesuaikan tegangan yang dibutuhkan relay
5	LCD 16x2 + I2C	1	Menampilkan level air dan status sistem
6	LED Indikator (Merah, Kuning, Hijau)	3	Menunjukkan level air (Low, Medium, High)
7	Resistor 220Ω	4	Untuk mencegah LED rusak
8	Buzzer	1	Alarm saat air penuh
9	Led Indikator Putih	1	Simulasi pengisian air (pompa air)
10	Breadboard/Protoboard	1	Tempat perakitan rangkaian
11	Kabel Jumper (Male-Female, Male-Male)	±20	Penghubung antar komponen
12	Adaptor 5V / USB Power Supply	1	Sumber daya rangkaian
13	Laptop / PC	1	Untuk upload program ke ESP8266
14	Aplikasi Blynk (Android/iOS)	1	Monitoring dan kontrol jarak jauh

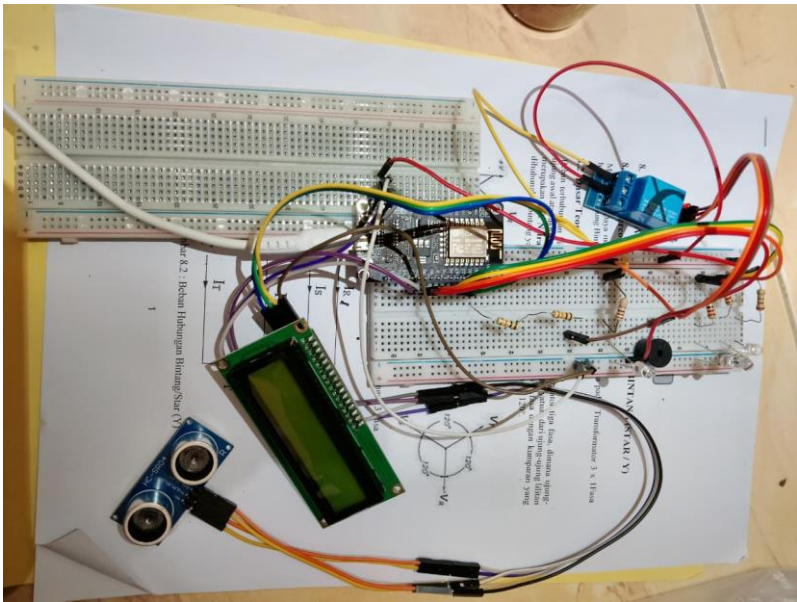
IV. LANGKAH PERCOBAAN

1. Siapkan semua alat dan bahan yang dibutuhkan.
2. Hubungkan Rangkaian seperti tabel dibawah ini.

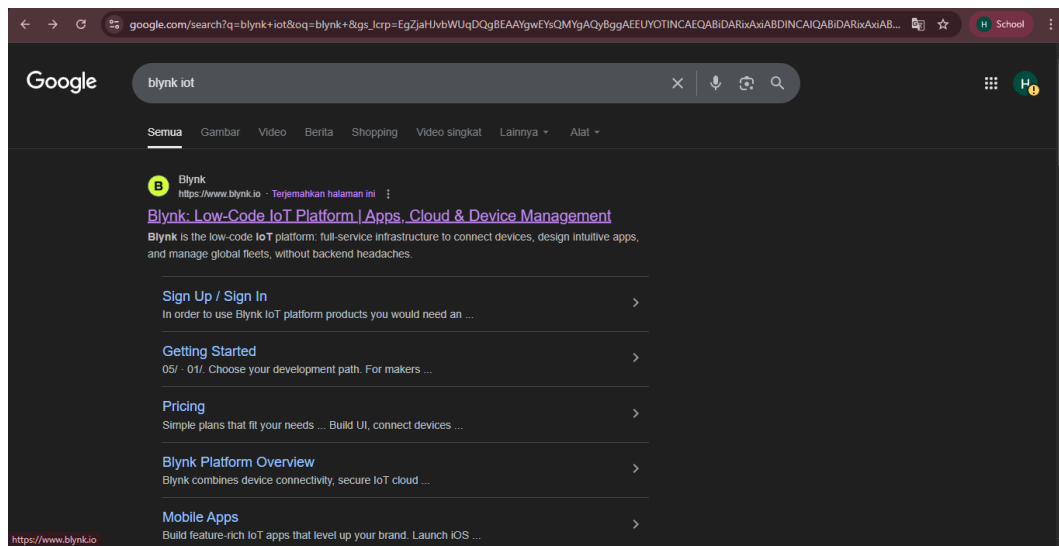
Komponen	Pin Komponen	Ke NodeMCU	Catatan
HC-SR04	VCC	VIN (5V)	HC-SR04 butuh 5V.
	GND	GND	Ground disatukan.
	TRIG	D7	OUTPUT 3.3V dari ESP → aman.
	ECHO	D8	Wajib turunkan ke 3.3V (divider 10k : 15k → ~3.0V atau 1k : 2k). Tanpa ini berisiko merusak pin.
LCD 16×2 + I2C (0x27)	VCC	VIN (5V)	Backlight stabil di 5V.
	GND	GND	—

	SDA	D2 (GPIO4)	I ² C default ESP8266. Saran: gunakan level shifter I ² C (karena pull-up backpack biasanya ke 5V).
	SCL	D1 (GPIO5)	I ² C default.
LED LOW	Anoda	D3 via resistor 220Ω ke anoda	Katoda → GND.
	Katoda	GND	—
LED MEDIUM	Anoda	D4 via 220Ω	Katoda → GND.
	Katoda	GND	—
LED HIGH	Anoda	D5 via 220Ω	Katoda → GND.
	Katoda	GND	—
Buzzer (aktif)	+	D6	Pakai buzzer modul aktif kecil (≤ 20 mA). Jika buzzer 5V/arus besar, pakai transistor NPN + diode flyback.
	—	GND	—
Relay 1-CH (logika)	IN	D0 via 20K Ω	Aktif-LOW (LOW = relay ON).
	VCC	VIN (5V)	—
	GND	GND	—
Relay (kontak beban)	COM	Sumber beban (mis. +12V DC / L AC)*	Kontak daya terpisah dari logika.
	NO	Ke beban (pompa/LED simulasi)	NO tertutup saat relay ON.

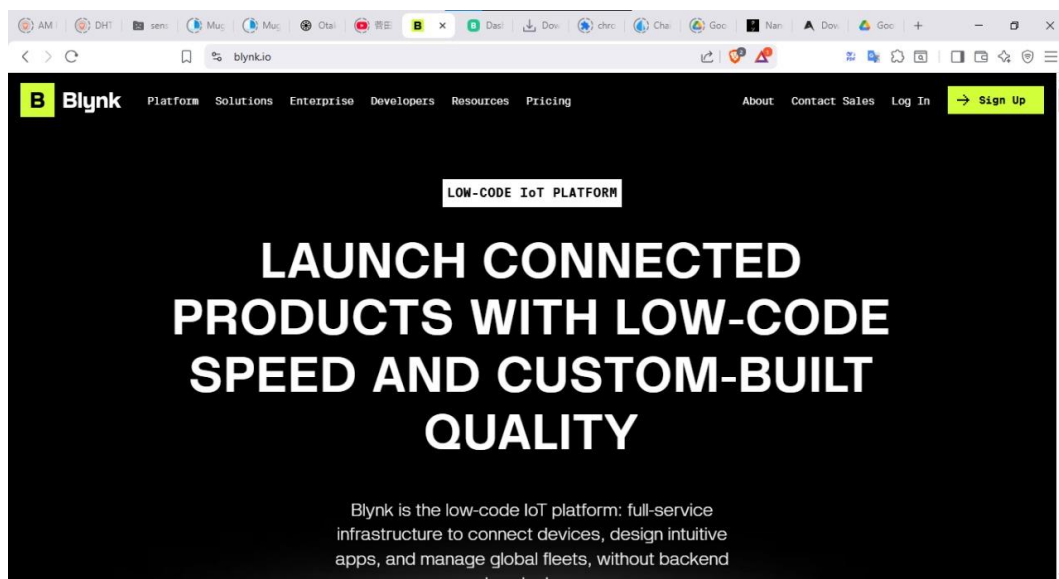
LED putih (simulasi pompa air) + resistor 220Ω di jalur output relay NO–COM



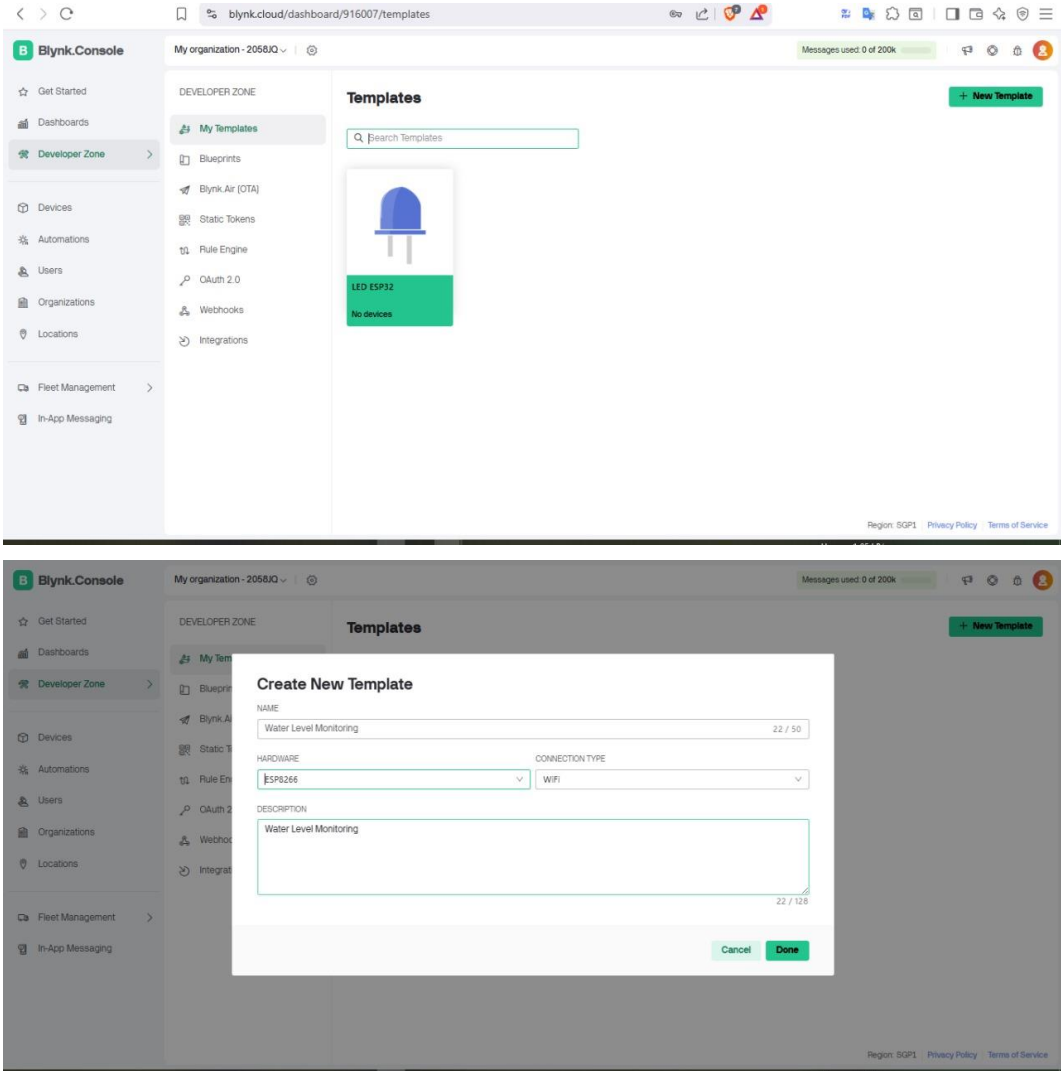
3. Sekarang, mari kita siapkan web Blynk langkah demi langkah. Untuk itu, ikuti petunjuk di bawah ini.
- 3.1. Kunjungi situs web Blynk dan buat akun baru menggunakan alamat email Anda. Kemudian, masuk ke akun Anda dan buat templat baru untuk proyek ini.



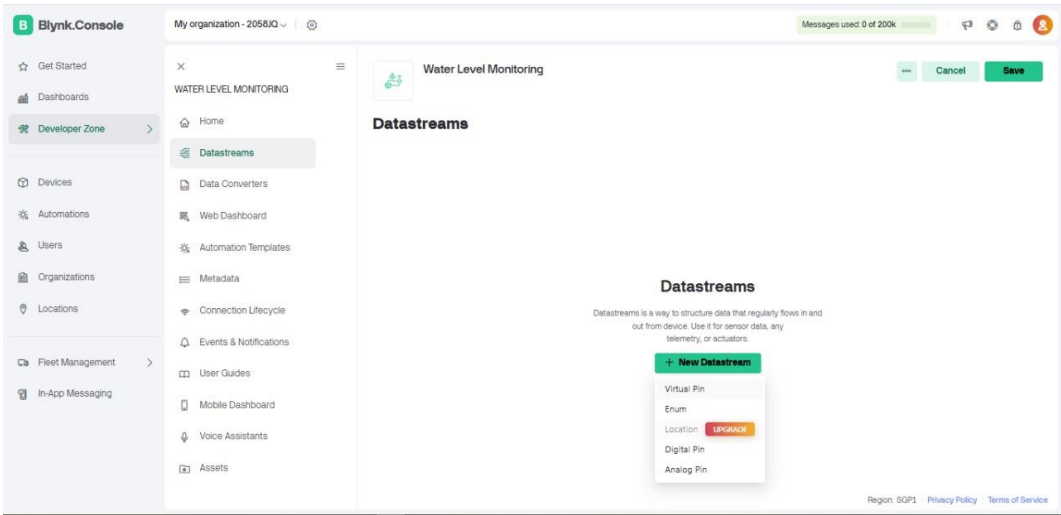
- 3.2. Jika anda sudah mempunyai akun silahkan Log In, jika belum mempunyai akun silahkan Sign In untuk membuat akun baru.



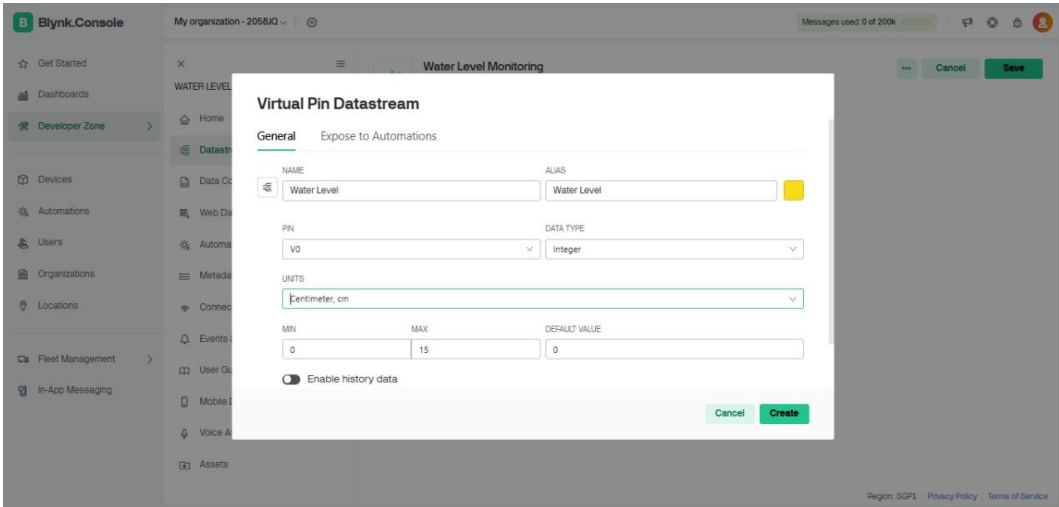
3.3.Klik New Tempate dan isi sesuai dalam gambar.



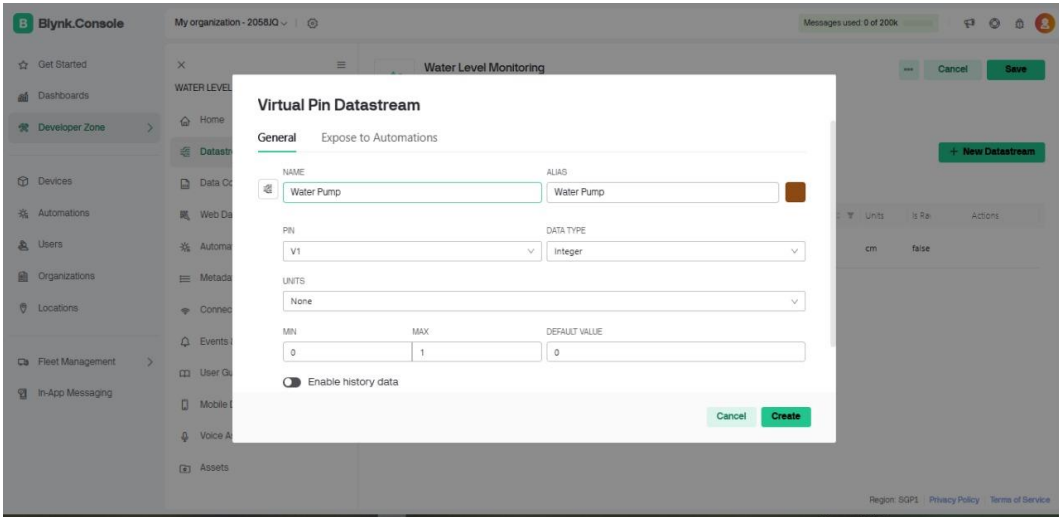
3.4.Klik +New Data Stream dan pilih Virtual Pin



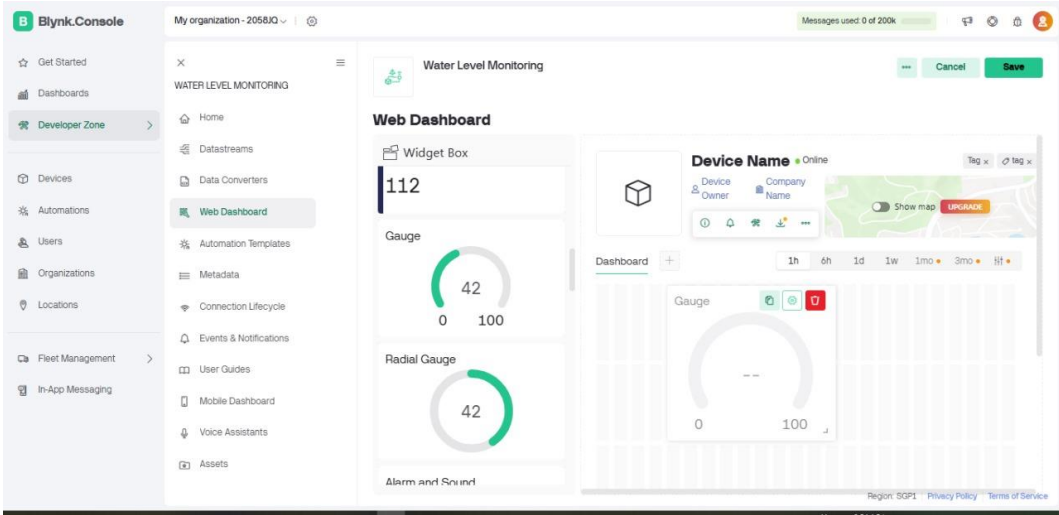
3.5.Isi data sesuai dalam gambar, Datastream dibuat untuk indikator water level.



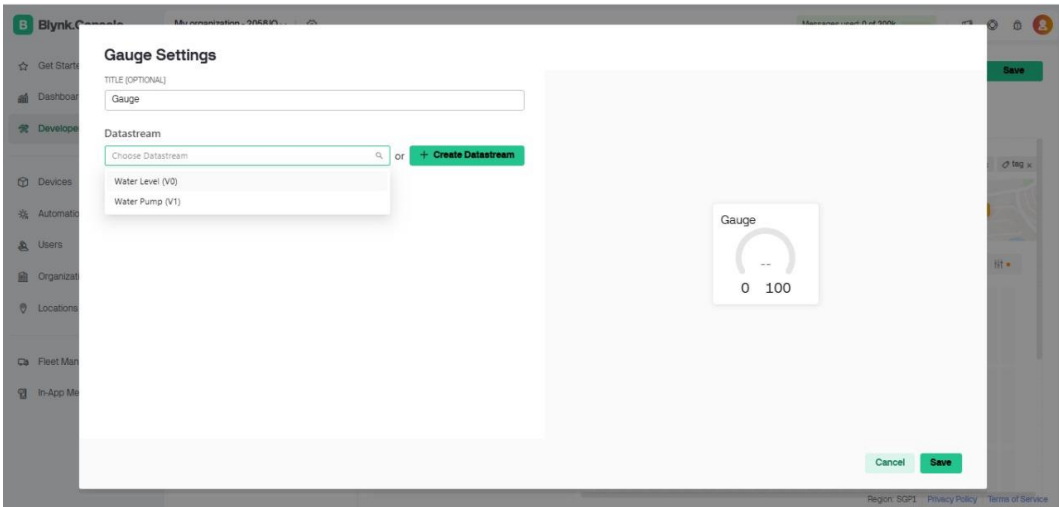
3.6.Buat lagi untuk switch water pump (pompa air).



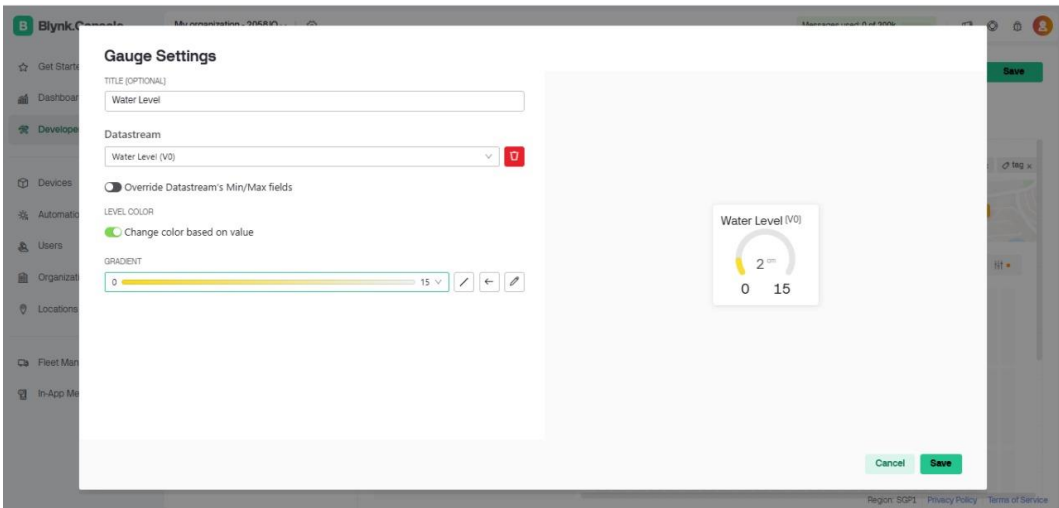
3.7.Klik Bagian Web Dashboard yang nantinya akan ditampilkan dalam interface saat program berjalan, pilih gauge dan edit.



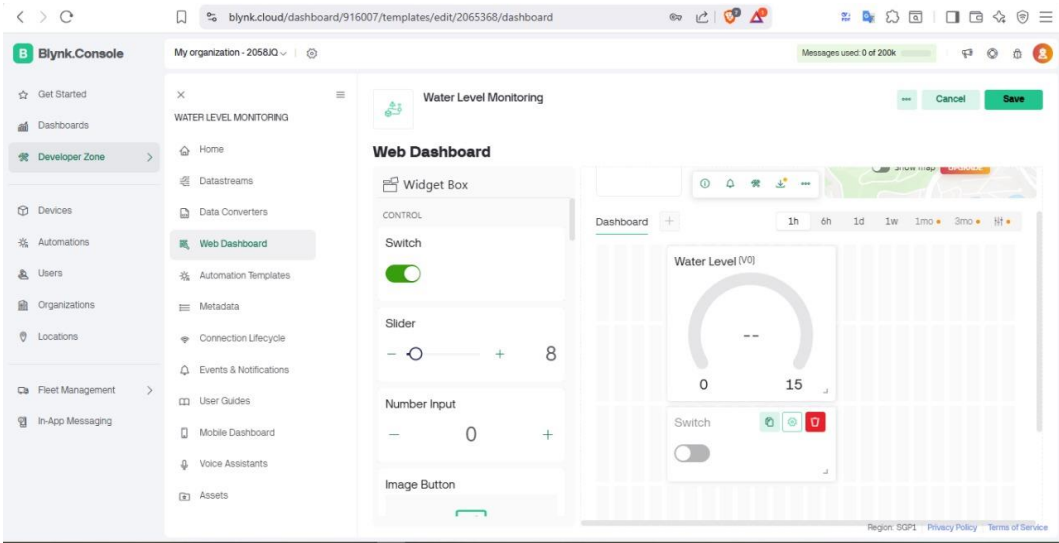
3.8.Untuk datastream pilih Water Level (V0).



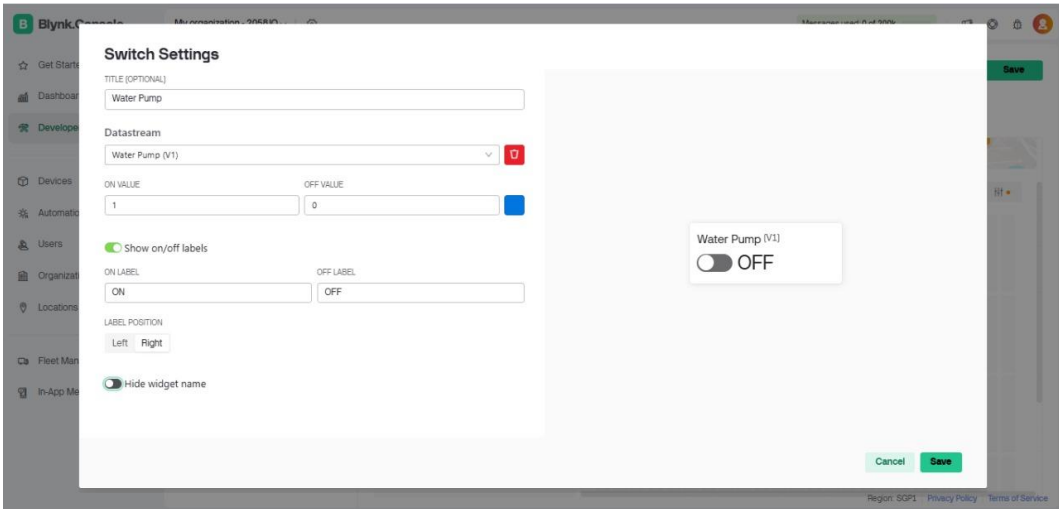
3.9.Edit untuk mempercantik tampilan.



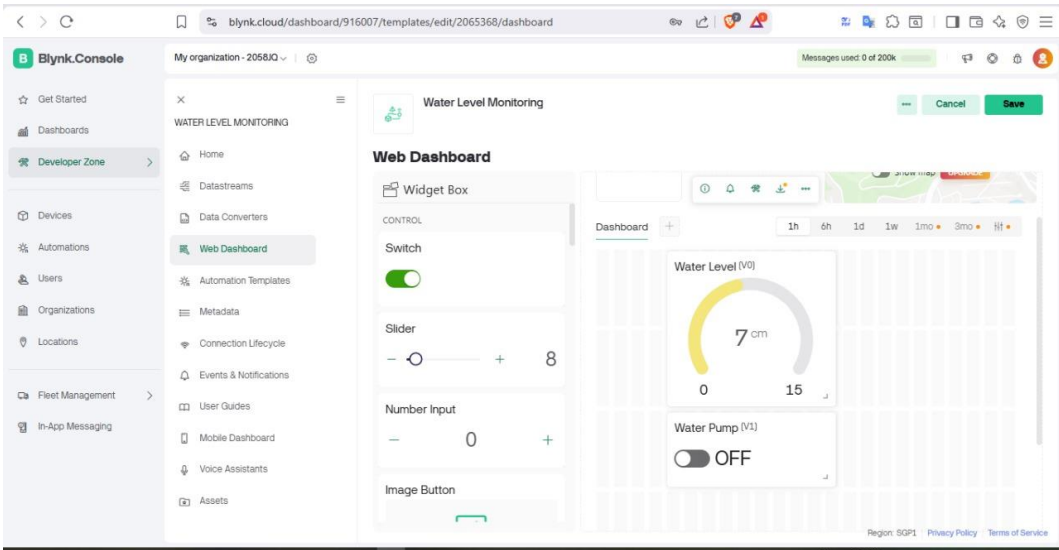
3.10.Lanjut pilih Switch dan edit.



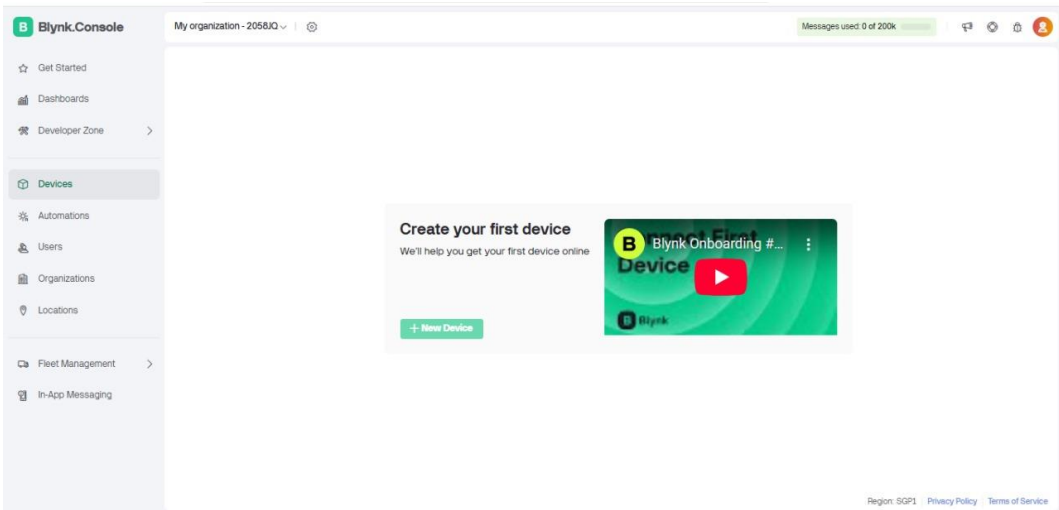
3.11. Pilih dan isi data sesuai gambar dibawah.



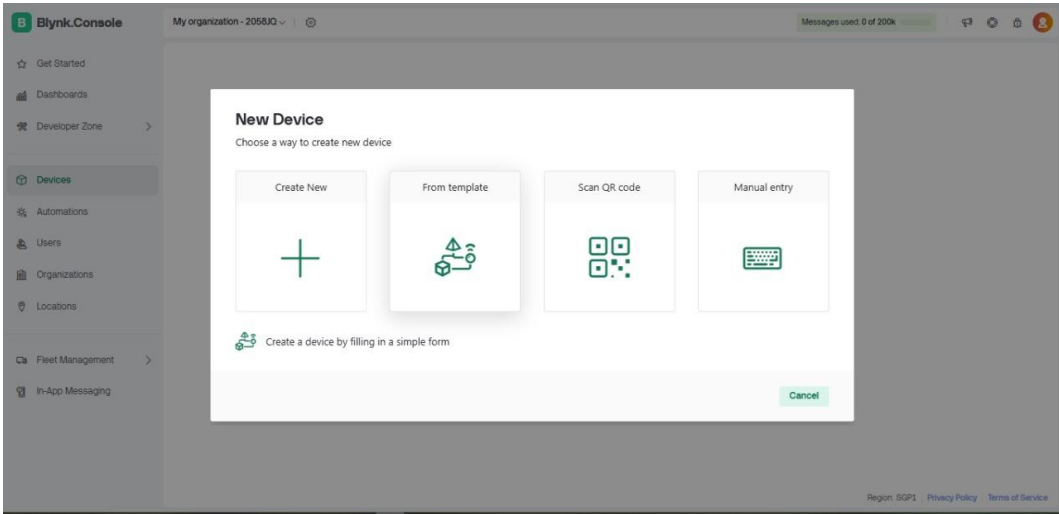
3.12. Jika semuanya sudah selesai klik save.



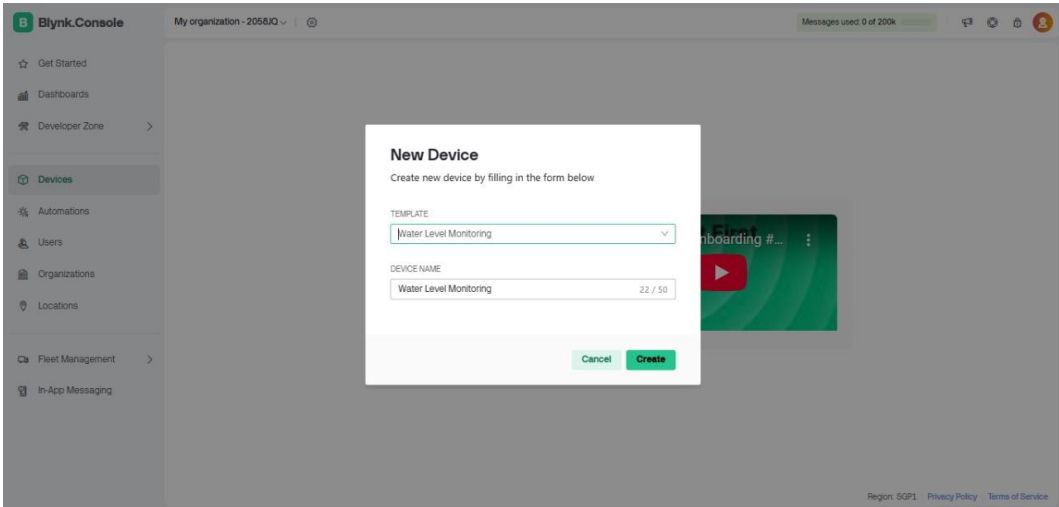
3.13. Klik +New Device.



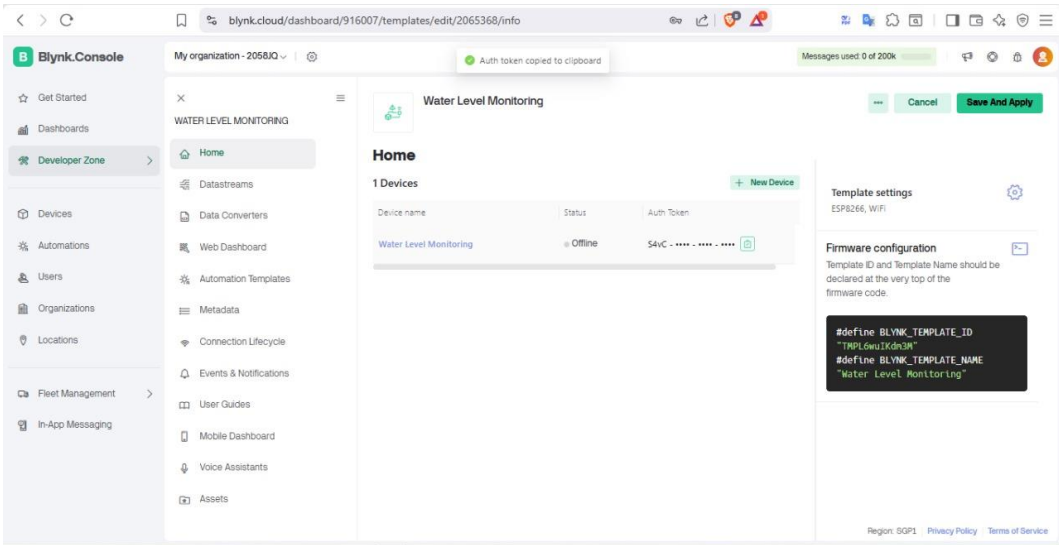
3.14. Klik from template.



3.15. Pilih Template yang tadi.

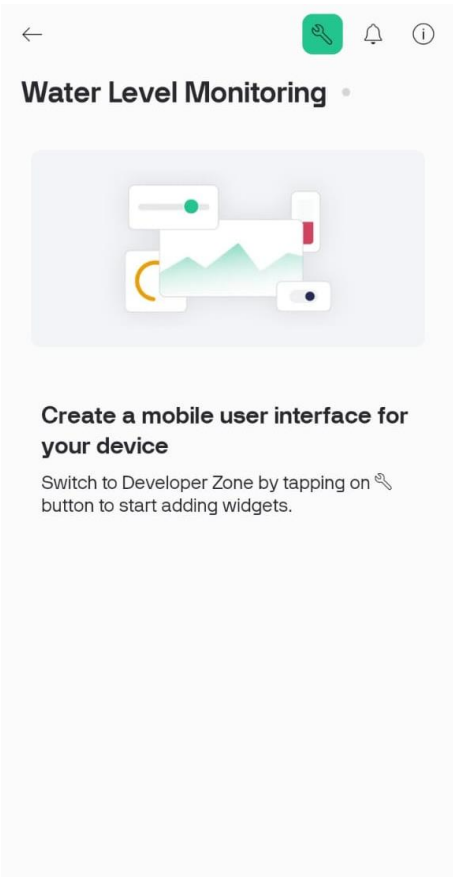
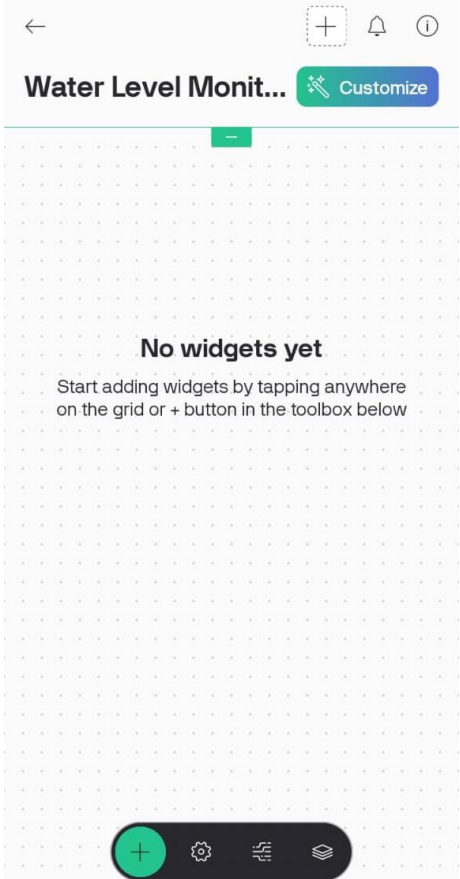
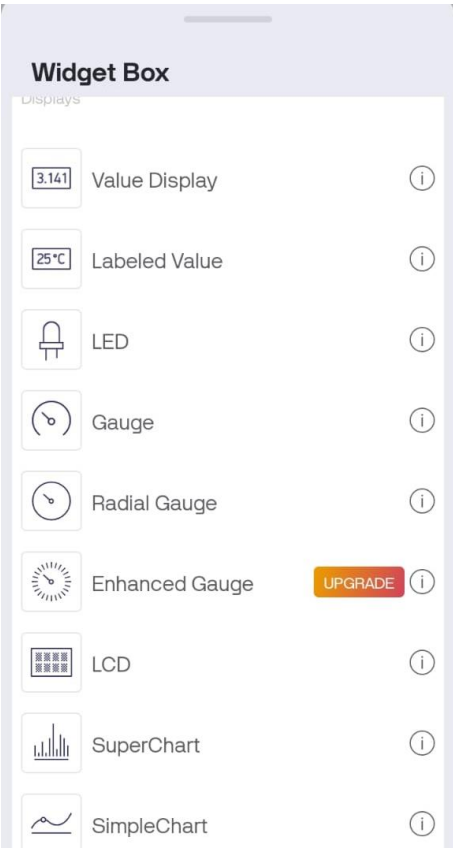
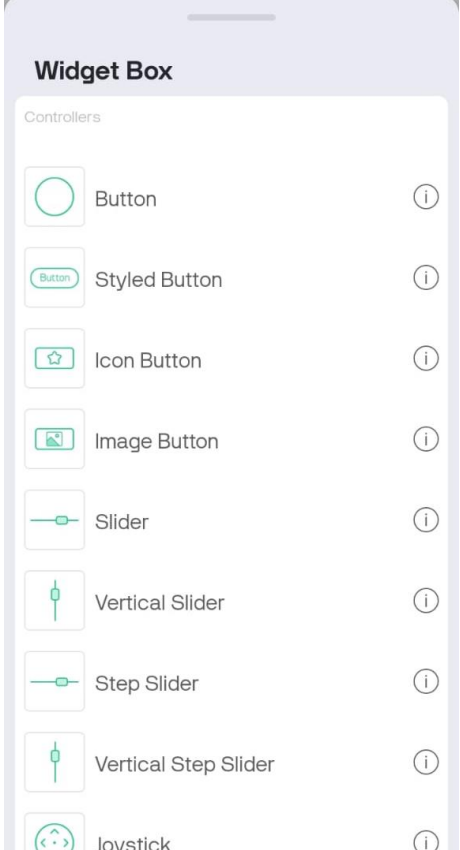


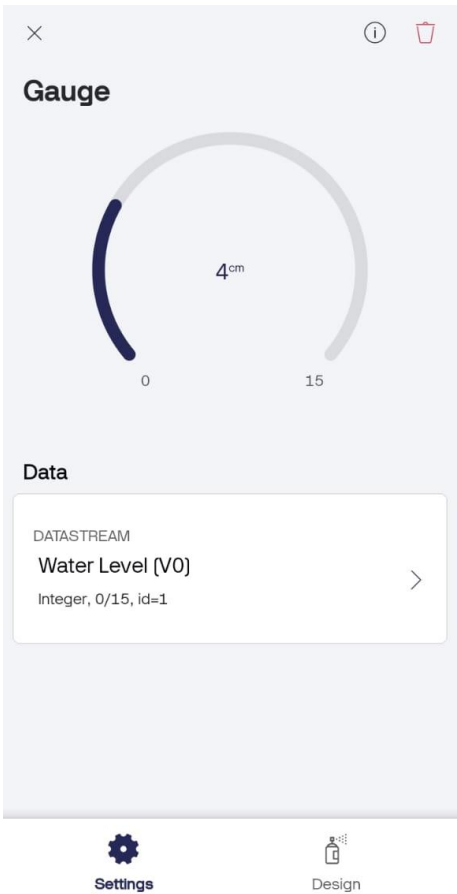
3.16. Klik Developer Zone lalu ke home, Terdapat Auth Token yang nantinya akan ditempel ke program.



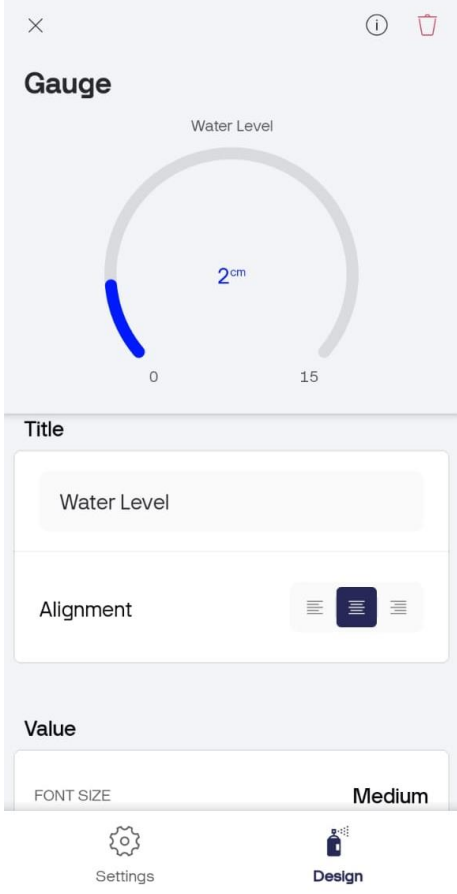
4. Sekarang, mari kita siapkan dasbor seluler Blynk untuk proyek ini. Untuk itu, ikuti petunjuk di bawah ini.

5.1 Unduh dan instal aplikasi Blynk di ponsel pintar Anda. Kemudian, Log In menggunakan akun yang tadi. Ikuti langkah seperti dalam tabel.

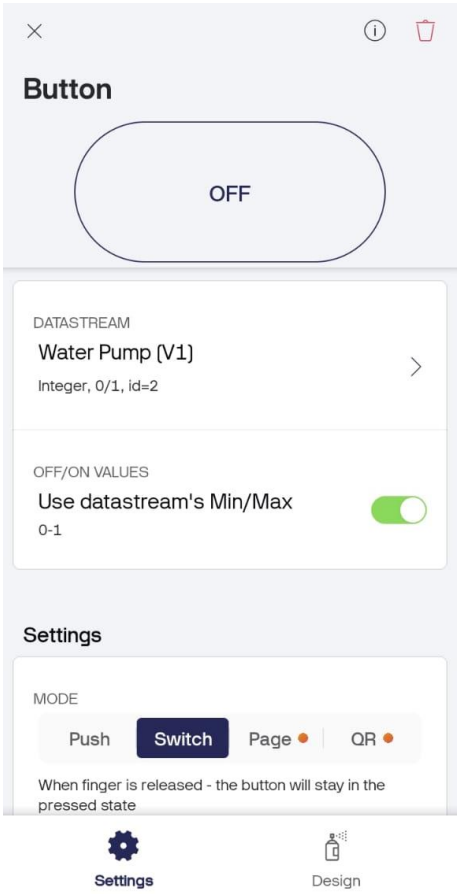
	
5.2 Klik icon kunci untuk membuat widgets	5.3 Klik tombol tambah
	
5.4 Tambahkan Gauge dan taruh sesuai selera	5.5 Tambahkan button



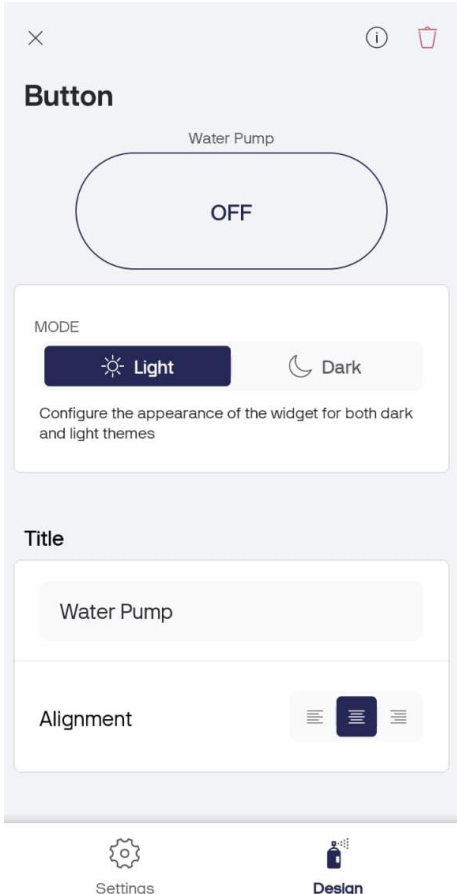
5.6 Untuk Gauge pilih Datastream V0



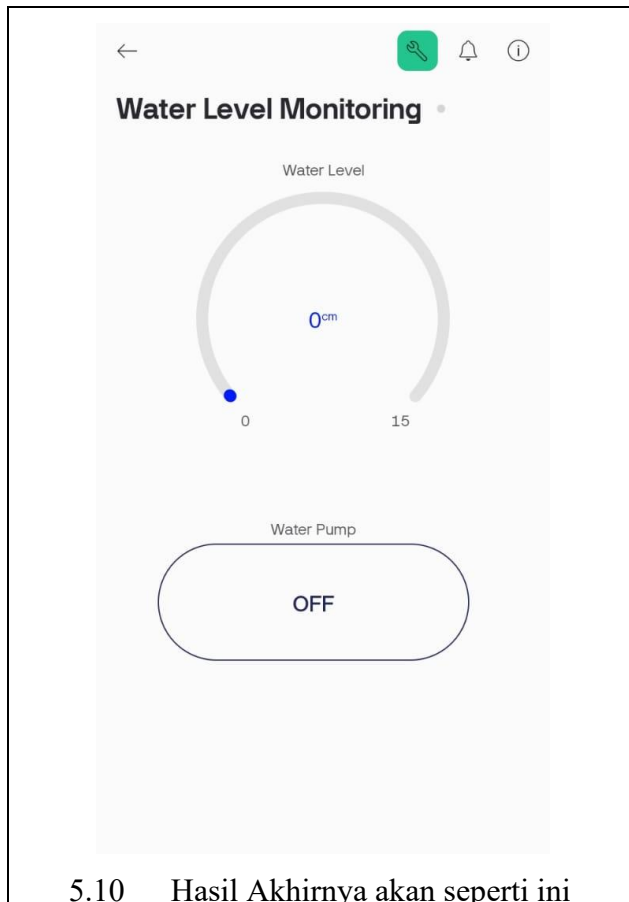
5.7 Edit seperti gambar/sesuai keinginan dengan Title Water Level



5.8 Untuk Button pilih Datastream V1,
Ganti Mode Push ke Switch



5.9 Edit seperti gambar/sesuai keinginan dengan Title Water Pump



5.10 Hasil Akhirnya akan seperti ini

5. Hubungkan USB NodeMCU ESP8266 ke komputer dan unggah program untuk sistem ini.

```

1)  /* Water Level Monitoring ESP8266 + Blynk + LCD
2)      Dengan Kalibrasi HC-SR04
3)      -----
4)      - Jarak (cm) = (t_us / 58.3) -> lalu
      dikalibrasi: y = a*x + b
5)      - Level:
6)          LOW    -> D3
7)          MEDIUM -> D3 + D4
8)          HIGH   -> D3 + D4 + D5
9)          FULL   -> D3 + D4 + D5 + BUZZER(D6) berkedip
10)     - Relay D0 (aktif-LOW)
11)  */
12)
13)  #include <LiquidCrystal_I2C.h>
14)  #define BLYNK_PRINT Serial
15)  #include <ESP8266WiFi.h>
16)  #include <BlynkSimpleEsp8266.h>
17)
18)  LiquidCrystal_I2C lcd(0x27, 16, 2);
19)
20)  // --- Blynk / WiFi ---
21)  char auth[] = "S4vCWAgqU_KuzDDeAFYSyO-gEUsmu6Wo";
22)  char ssid[] = "HiFi-13897";
23)  char pass[] = "pssslemanale";
24)  BlynkTimer timer;
25)
26)  // --- Pin ---
27)  #define TRIG    D7
28)  #define ECHO    D8
29)  #define LED3    D3      // LOW

```



```

30) #define LED4    D4      // MEDIUM/HIGH/FULL
31) #define LED5    D5      // HIGH/FULL
32) #define BUZZER D6      // buzzer
33) #define RELAY   D0      // aktif-LOW (LOW=ON)
34)
35) // --- Kalibrasi HC-SR04 (dari regresi:  $y = a*x + b$ ) ---
36) const float CAL_A = 0.9889f; // slope
37) const float CAL_B = 0.00f;   // offset (ubah jika
    ada)
38)
39) // --- Skala tinggi air ---
40) const int MaxLevel = 15;      // cm
    (0..15 cm)
41) const float T_LOW  = MaxLevel * 0.33f; // ≈5 cm
42) const float T_HIGH = MaxLevel * 0.66f; // ≈10 cm
43) const float T_FULL = MaxLevel * 0.90f; // ≈13.5
    cm
44)
45) bool relayOn = false;
46) bool fullAlarm = false, blinkState = false;
47) unsigned long lastBlink = 0;
48) const unsigned long BLINK_MS = 300;
49)
50) int levelCm = 0;             // tinggi air (cm)
    terkalibrasi
51) String levelText = "---";
52)
53) //
    =====
54) // Baca jarak terkalibrasi (cm) dari HC-SR04
55) float readDistanceCmCal() {
56)     // trigger 10 μs
57)     digitalWrite(TRIG, LOW); delayMicroseconds(4);
58)     digitalWrite(TRIG, HIGH); delayMicroseconds(10);
59)     digitalWrite(TRIG, LOW);
60)
61)     // durasi ECHO (μs), timeout 30 ms
62)     unsigned long t = pulseIn(ECHO, HIGH, 30000UL);
63)     if (t == 0) return MaxLevel + 200; // jika
    timeout
64)
65)     // jarak mentah (cm) pakai fisika:  $t/58.3$ 
66)     float d_raw = t / 58.3f;
67)
68)     // terapkan kalibrasi:  $y = a*x + b$ 
69)     float d_cal = CAL_A * d_raw + CAL_B;
70)     return d_cal;
71) }
72)
73) void lcdShow() {
74)     // Baris 1
75)     lcd.setCursor(0, 0);
76)     lcd.print("WLevel ");
77)     lcd.print(levelText);

```

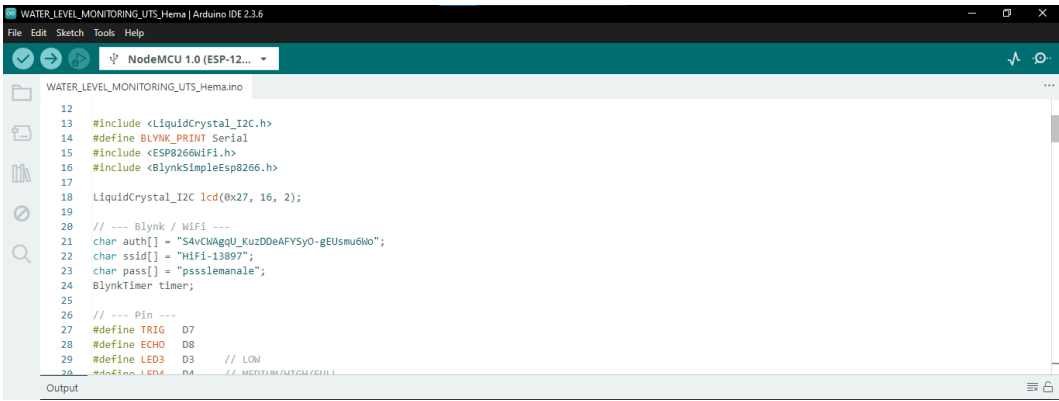
```
78)   lcd.print("    "); // bersihkan sisa
79)
80)   // Baris 2
81)   lcd.setCursor(0, 1);
82)   lcd.print("h:");
83)   if (levelCm < 10) lcd.print('0');
84)   lcd.print(levelCm);
85)   lcd.print("cm Motor:");
86)   lcd.print(relayOn ? "ON " : "OFF");
87)   lcd.print(" ");
88) }
89)
90) void setLEDs(bool L3, bool L4, bool L5) {
91)   digitalWrite(LED3, L3);
92)   digitalWrite(LED4, L4);
93)   digitalWrite(LED5, L5);
94) }
95)
96) void handleFullBlink() {
97)   if (!fullAlarm) return;
98)   if (millis() - lastBlink >= BLINK_MS) {
99)     lastBlink = millis();
100)    blinkState = !blinkState;
101)    digitalWrite(LED3, blinkState);
102)    digitalWrite(LED4, blinkState);
103)    digitalWrite(LED5, blinkState);
104)    digitalWrite(BUZZER, blinkState);
105)  }
106) }
107)
108) //
=====
109) void ultrasonicTask() {
110)   float distance =
       readDistanceCmCal();           // jarak sensor →
       cm (terkalibrasi)
111)   int height = (distance <= MaxLevel) ? (MaxLevel -
(int)(distance + 0.5f)) : 0;
112)   if (height < 0) height = 0;
113)   if (height > MaxLevel) height = MaxLevel;
114)
115)   levelCm = height;
116)   Blynk.virtualWrite(V0, levelCm);
117)
118)   // Tentukan status level
119)   if (levelCm >= (int)T_FULL) {
120)     levelText = "Full ";
121)     fullAlarm = true;
122)   } else if (levelCm >= (int)T_HIGH) {
123)     levelText = "High ";
124)     fullAlarm = false;
125)     setLEDs(HIGH, HIGH, HIGH);
126)     digitalWrite(BUZZER, LOW);
127)   } else if (levelCm >= (int)T_LOW) {
128)     levelText = "Medium";
```

```
129)     fullAlarm = false;
130)     setLEDs(HIGH, HIGH, LOW);
131)     digitalWrite(BUZZER, LOW);
132) } else {
133)     levelText = "Low  ";
134)     fullAlarm = false;
135)     setLEDs(HIGH, LOW, LOW);
136)     digitalWrite(BUZZER, LOW);
137) }
138)
139) if (fullAlarm) handleFullBlink();
140) lcdShow();
141) }
142)
143) //
=====
144) BLYNK_WRITE(V1) {
145)     bool v = param.asInt();
146)     if (v) { digitalWrite(RELAY, LOW); relayOn =
        true; }
147)     else { digitalWrite(RELAY, HIGH); relayOn =
        false; }
148)     lcdShow();
149) }
150)
151) //
=====
152) void setup() {
153)     Serial.begin(9600);
154)     lcd.init(); lcd.backlight();
155)
156)     pinMode(TRIG, OUTPUT);
157)     pinMode(ECHO, INPUT);          // ECHO wajib via
        divider ke 3.3V saat wiring nyata
158)     pinMode(LED3, OUTPUT);
159)     pinMode(LED4, OUTPUT);
160)     pinMode(LED5, OUTPUT);
161)     pinMode(BUZZER, OUTPUT);
162)     pinMode(RELAY, OUTPUT);
163)
164)     digitalWrite(RELAY, HIGH); relayOn = false;
165)     setLEDs(LOW, LOW, LOW);
166)     digitalWrite(BUZZER, LOW);
167)
168)     Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
169)
170)     lcd.clear();
171)     lcd.setCursor(0,0); lcd.print("Water Level
        Init");
172)     lcd.setCursor(0,1); lcd.print("Motor: OFF");
173)     delay(1000);
174)     lcd.clear();
175)
176)     timer.setInterval(300L, ultrasonicTask);
177) }
```

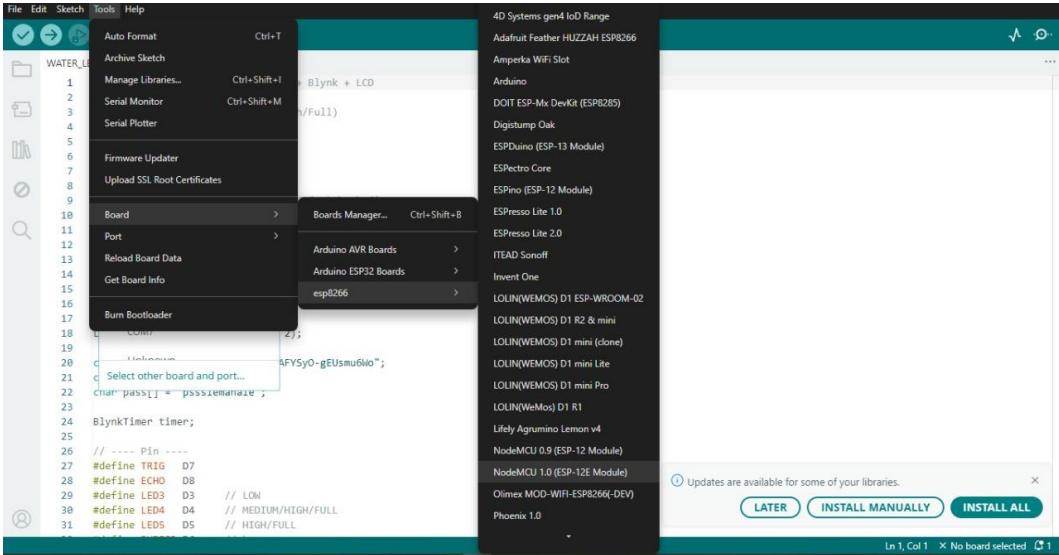
```
178)
179) //
=====
180) void loop() {
181)   Blynk.run();
182)   timer.run();
183)   handleFullBlink();
184) }
```

6. Masukkan Auth Token (yang terdapat pada langkah 4.16), Wi-Fi dan Password anda kedalam program.

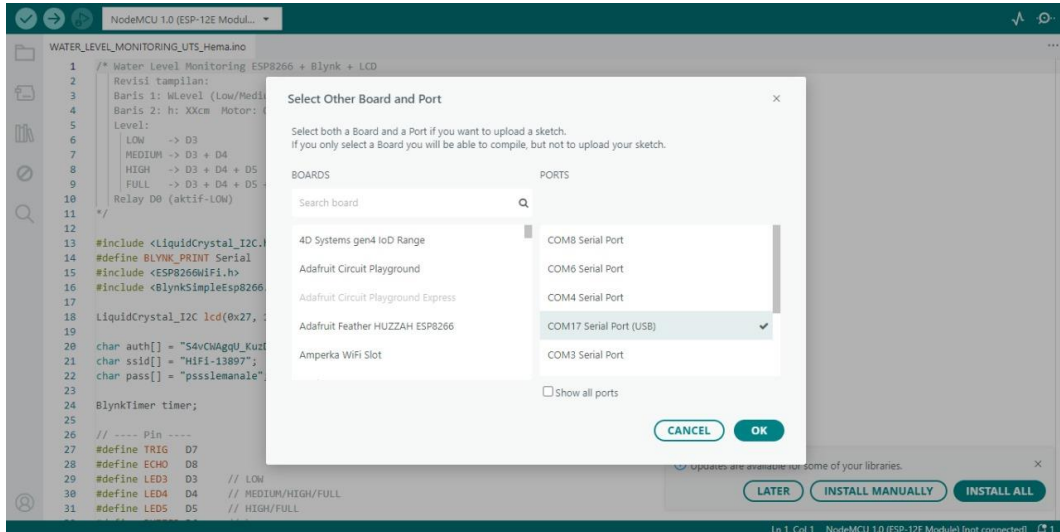
```
20) char auth[] = "S4vCWAgqU_KuzDDeAFYSyO-gEUsmu6Wo";
21) char ssid[] = "HiFi-13897";
22) char pass[] = "pssselemanale";
```



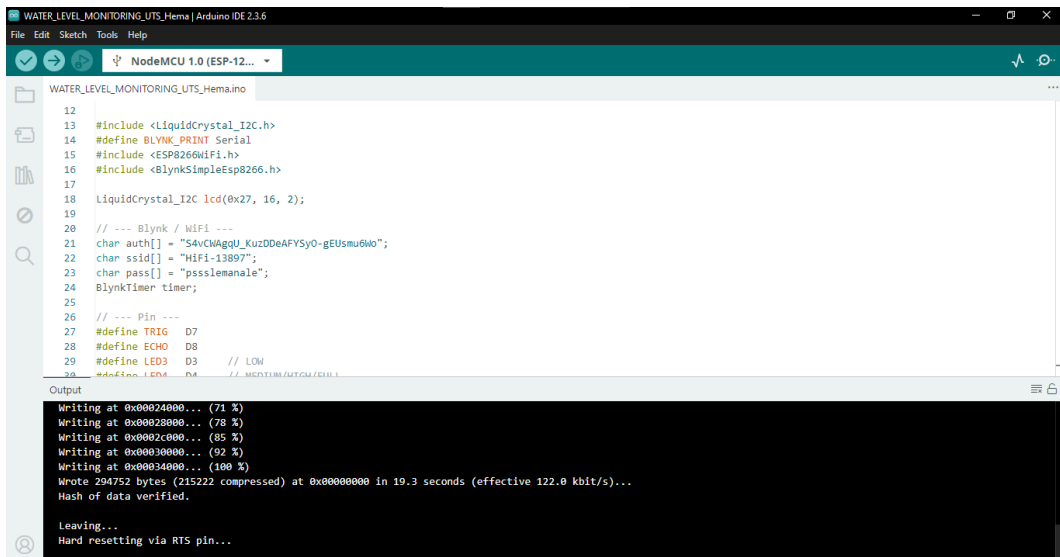
7. Pilih Board yang anda gunakan seperti gambar.



8. Pilih port USB yang tersambung ke komputer anda.



9. Jalankan Program dan tunggu sampai selesai.



V. Analisis

Sebelum melakukan percobaan tersebut kita harus kalibrasi terlebih dahulu dengan program berikut

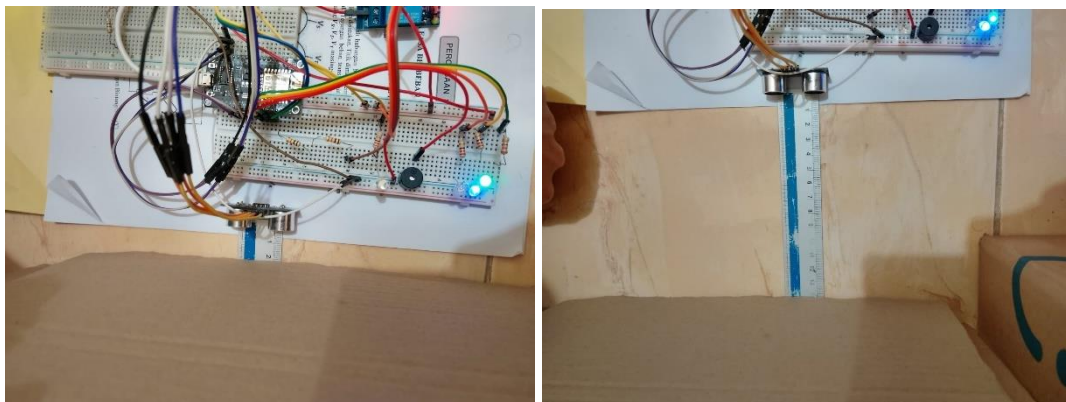
```
1) /* HC-SR04 Distance Measurement (ESP8266)
2) -----
3) Mengukur jarak berdasarkan durasi ECHO (µs)
4) Rumus: jarak (cm) = t / 58.3
5) Output: 1 data per detik, berhenti setelah 10
   detik
6) */
7)
8) #define TRIG D7
9) #define ECHO D8
10)
11) unsigned long startTime; // waktu mulai program
12) const unsigned long runTime = 10000; // 10 detik
   (ms)
13)
14) void setup() {
15)   Serial.begin(9600);
16)   pinMode(TRIG, OUTPUT);
```

```

17) pinMode(ECHO, INPUT);
18) digitalWrite(TRIG, LOW);
19) delay(1000);
20)
21) Serial.println("Mulai pengukuran jarak HC-SR04
    (10 detik)...");
22)   startTime = millis();
23) }
24)
25) void loop() {
26)   if (millis() - startTime >= runTime) {
27)     Serial.println("Pengukuran selesai.");
28)     while (true); // berhenti
29)   }
30)
31)   // --- Trigger 10 µs ---
32)   digitalWrite(TRIG, LOW);
33)   delayMicroseconds(4);
34)   digitalWrite(TRIG, HIGH);
35)   delayMicroseconds(10);
36)   digitalWrite(TRIG, LOW);
37)
38)   // --- Baca durasi echo ---
39)   unsigned long duration = pulseIn(ECHO, HIGH,
    30000UL); // timeout 30 ms
40)
41)   // --- Hitung jarak (cm) ---
42)   float distance = duration / 58.3; // rumus
    fisika: v = 343 m/s
43)
44)   // --- Tampilkan hasil ---
45)   Serial.print("Durasi: ");
46)   Serial.print(duration);
47)   Serial.print(" us | Jarak: ");
48)   Serial.print(distance, 2);
49)   Serial.println(" cm");
50)
51)   delay(1000); // 1 data per detik
52) }

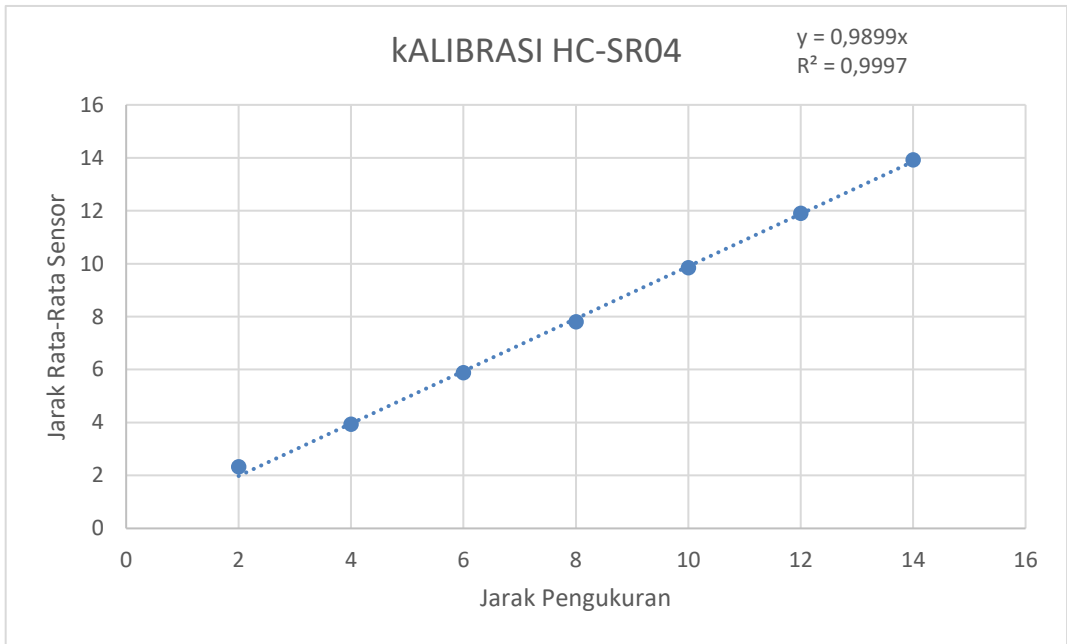
```

Urut dari jarak 2 cm (kelipatan 2) sampai 14 cm



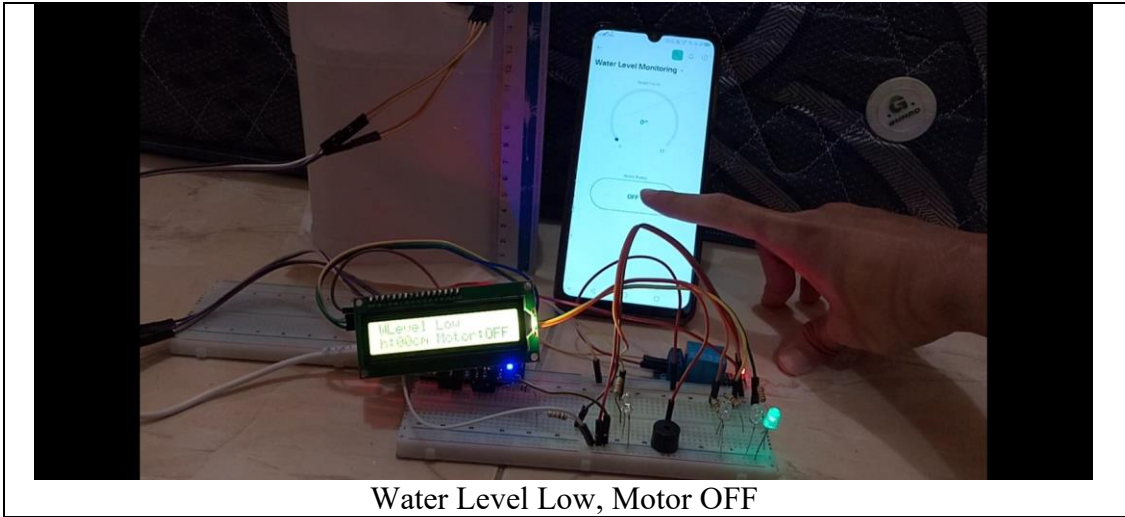
Diperoleh hasil

No	Pengukur an 1 (cm)	Pengukur an 2 (cm)	Pengukur an 3 (cm)	Pengukur an 4 (cm)	Pengukur an 5 (cm)	Pengukur an 6 (cm)	Pengukur an 7 (cm)
1	2,37	3,98	5,93	7,89	9,91	11,97	14
2	2,32	3,95	5,87	7,8	9,85	11,89	13,91
3	2,32	3,95	5,85	7,8	9,83	11,89	13,91
4	2,32	3,91	5,87	7,8	9,85	11,9	13,91
5	2,32	3,95	5,87	7,8	9,85	11,9	13,91
6	2,32	3,95	5,87	7,8	9,83	11,89	13,95
7	2,32	3,95	5,87	7,8	9,83	11,89	13,95
8	2,32	3,95	5,87	7,8	9,85	11,89	13,91
Rata- rata	2,33	3,94	5,88	7,81	9,85	11,9	13,92
Aktua l	2	4	6	8	10	12	14
Error %	16,5 %	1,5 %	2,0 %	2,4 %	1,5 %	0,8 %	0,6 %
Total Error %	3,6 %						



Masukkan $y = 0,9899x$ kedalam program (program yang tadi telah dimasukkan hasil kalibrasi)

VI. HASIL PERCOBAAN





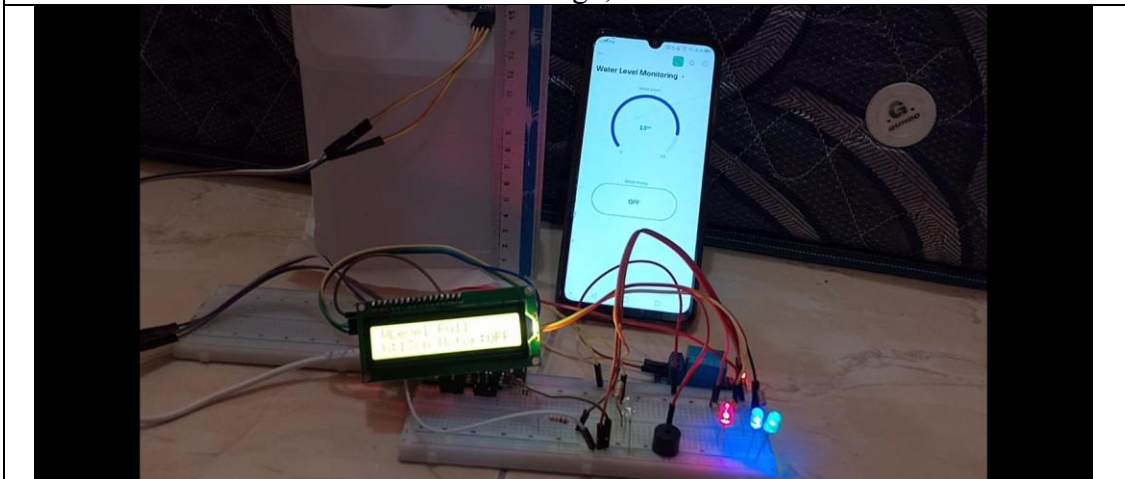
Water Level Low, Motor ON



Water Level Medium, Motor ON



Water Level High, Motor ON



Water Level Full, Motor OFF, LED dan buzzer ON berkedip

Cara kerja program tersebut:

1. Inisialisasi

- Inisialisasi LCD I²C (0x27), Wi-Fi+Blynk, dan timer.
- Set pin: TRIG=D7, ECHO=D8, LED3=D3, LED4=D4, LED5=D5, BUZZER=D6, RELAY=D0 (aktif-LOW).
- Relay dimatikan default (HIGH = OFF), semua LED & buzzer OFF.
- `timer.setInterval(300 ms, ultrasonicTask)` → fungsi utama sensor dieksekusi tiap 0,3 s.

2. Pembacaan jarak (HC-SR04)

Fungsi `readDistanceCmCal()` bekerja dengan langkah berikut:

1. Kirim pulsa TRIG 10 μ s.
2. Baca durasi HIGH pin ECHO dengan `pulseIn()` (μ s).
3. Konversi ke jarak mentah:
 $d_raw = t_echo / 58.3 \text{ (cm)}$
4. Terapkan kalibrasi linear dari hasil percobaan:
 $d_cal = CAL_A * d_raw + CAL_B$
(default: $CAL_A=0.9889$, $CAL_B=0.00$ — bisa diubah sesuai regresi terbaru).
5. Return `d_cal`. Jika timeout (tak ada echo), kembalikan nilai besar agar dianggap “di luar tangki”.

3. Konversi ke tinggi air

- Tangki dianggap $MaxLevel = 15 \text{ cm}$ (0–15 cm).
- Hitung tinggi air:
 $height = MaxLevel - d_cal$
- Nilai dibatasi antara 0 dan 15 cm.
- Simpan ke `levelCm` dan kirim ke Blynk V0 untuk tampilan gauge.

4. Penentuan status level

Ambang batas berdasarkan tinggi air:

- LOW : $h < 0.33 \cdot Max$ ($\approx < 5 \text{ cm}$)
- MEDIUM : $0.33 - 0.66 \cdot Max$ ($\approx 5 - 10 \text{ cm}$)
- HIGH : $0.66 - 0.90 \cdot Max$ ($\approx 10 - 13 \text{ cm}$)
- FULL : $h \geq 0.90 \cdot Max$ ($\geq 13 \text{ cm}$)

Efek ke indikator:

- LOW → LED3 ON; LED4, LED5, buzzer OFF.
- MEDIUM → LED3+LED4 ON; LED5, buzzer OFF.
- HIGH → LED3+LED4+LED5 ON; buzzer OFF.
- FULL → semua LED berkedip + buzzer berkedip ($BLINK_MS=300 \text{ ms}$).

5. Tampilan LCD

- Baris 1: “WLevel <Low/Medium/High/Full>” (dibersihkan sisa karakter agar rapi).
- Baris 2: “h:XXcm Motor: ON/OFF” → XX adalah nilai `levelCm` (dua digit dengan leading zero).

6. Kendali Relay (manual via Blynk)

- V1 (Switch):
 - 1 → digitalWrite(RELAY, LOW) (ON, karena aktif-LOW)
 - 0 → digitalWrite(RELAY, HIGH) (OFF)
- Status relay disimpan di relayOn dan ditampilkan di LCD (Motor: ON/OFF).
- Versi ini tidak auto ON/OFF berdasarkan level air, relay dikendalikan manual oleh tombol Blynk V1.

7. Loop Utama

Loop utama menjalankan Blynk.run(), timer.run(), dan handleFullBlink() agar efek berkedip pada kondisi FULL tetap aktif dengan sinkronisasi Blynk.

VII. TUGAS

- 1) Buat program percobaan tadi menjadi ketika water level belum Full pompa air otomatis ON, jika water level Full maka pompa air otomatis OFF! (Indikator pompa air diganti simulasi LED putih)

Jawab :

Program :

```
1)  /* Water Level Monitoring ESP8266 + Blynk + LCD
2)     Versi Otomatis + Sinkronisasi Tombol Blynk
3)     -----
4)     - LOW  -> Relay ON  (pompa hidup)
5)     - FULL -> Relay OFF (pompa mati)
6)     - Tombol Blynk V1 ikut berubah otomatis
7)     - LCD: WLevel (Low/Medium/High/Full)
8)           h: XXcm  Motor: ON/OFF
9)     - 3 LED indikator (D3,D4,D5)
10)    - Buzzer (D6)
11)    - Relay aktif-LOW (D0)
12)  */
13)
14)  #include <LiquidCrystal_I2C.h>
15)  #define BLYNK_PRINT Serial
16)  #include <ESP8266WiFi.h>
17)  #include <BlynkSimpleEsp8266.h>
18)
19)  LiquidCrystal_I2C lcd(0x27, 16, 2);
20)
21)  char auth[] = "S4vCWAgqU_KuzDDDeAFYSyO-gEUsmu6Wo";
22)  char ssid[] = "HiFi-13897";
23)  char pass[] = "pssslemanale";
24)
25)  BlynkTimer timer;
26)
27)  // ---- Pin ----
28)  #define TRIG    D7
29)  #define ECHO    D8
30)  #define LED3    D3    // Low
31)  #define LED4    D4    // Medium
32)  #define LED5    D5    // High
33)  #define BUZZER  D6    // Full buzzer
```

```

34) #define RELAY D0      // Relay aktif-LOW
35)
36) // ---- Parameter ----
37) const int MaxLevel = 15;           // cm
38) const float T_LOW  = MaxLevel * 0.33f; // 5 cm
39) const float T_HIGH = MaxLevel * 0.66f; // 10 cm
40) const float T_FULL = MaxLevel * 0.90f; // 13.5 cm
41)
42) // ---- Variabel ----
43) bool relayOn = false;
44) bool fullAlarm = false, blinkState = false;
45) unsigned long lastBlink = 0;
46) const unsigned long BLINK_MS = 300;
47)
48) int levelCm = 0;
49) String levelText = "---";
50)
51) //
=====
52) // KONVERSI + KALIBRASI: jarak(cm) = 0.9889 * (t_us
   / 58.3)
53) int readDistanceCm() {
54)     digitalWrite(TRIG, LOW); delayMicroseconds(4);
55)     digitalWrite(TRIG, HIGH); delayMicroseconds(10);
56)     digitalWrite(TRIG, LOW);
57)     unsigned long t = pulseIn(ECHO, HIGH, 30000UL);
58)     if (t == 0) return MaxLevel + 200;
59)
60)     float d_raw = t / 58.3f;           // konversi
   fisika
61)     float d_cal = 0.9889f * d_raw;     // kalibrasi: y
   = 0.9889 x
62)     return (int)(d_cal + 0.5f);       // pembulatan
   ke cm
63) }
64)
65) //
=====
66) void lcdShow() {
67)     lcd.setCursor(0,0);
68)     lcd.print("WLevel ");
69)     lcd.print(levelText);
70)     lcd.print(" ");
71)
72)     lcd.setCursor(0,1);
73)     lcd.print("h:");
74)     if (levelCm < 10) lcd.print('0');
75)     lcd.print(levelCm);
76)     lcd.print("cm Motor:");
77)     lcd.print(relayOn ? "ON " : "OFF");
78)     lcd.print(" ");
79) }
80)
81) void setLEDs(bool L3, bool L4, bool L5) {
82)     digitalWrite(LED3, L3);

```

```
83)   digitalWrite(LED4, L4);
84)   digitalWrite(LED5, L5);
85) }
86)
87) void handleFullBlink() {
88)   if (!fullAlarm) return;
89)   if (millis() - lastBlink >= BLINK_MS) {
90)     lastBlink = millis();
91)     blinkState = !blinkState;
92)     digitalWrite(LED3, blinkState);
93)     digitalWrite(LED4, blinkState);
94)     digitalWrite(LED5, blinkState);
95)     digitalWrite(BUZZER, blinkState);
96)   }
97) }
98)
99) //
=====
100) void setRelayAuto(bool state) {
101)   digitalWrite(RELAY, state ? LOW : HIGH);
102)   relayOn = state;
103)   Blynk.virtualWrite(V1, state ? 1 : 0); // sinkron
      tombol
104) }
105)
106) //
=====
107) void ultrasonicTask() {
108)   int distance =
      readDistanceCm(); // sudah
      terkalibrasi
109)   levelCm = (distance <= MaxLevel) ? (MaxLevel -
      distance) : 0;
110)   Blynk.virtualWrite(V0, levelCm);
111)
112)   // Tentukan status air
113)   if (levelCm >= (int)T_FULL) {
114)     levelText = "Full ";
115)     fullAlarm = true;
116)     setRelayAuto(false); // OFF saat penuh
117)   }
118)   else if (levelCm >= (int)T_HIGH) {
119)     levelText = "High ";
120)     fullAlarm = false;
121)     setLEDs(HIGH, HIGH, HIGH);
122)     digitalWrite(BUZZER, LOW);
123)   }
124)   else if (levelCm >= (int)T_LOW) {
125)     levelText = "Medium";
126)     fullAlarm = false;
127)     setLEDs(HIGH, HIGH, LOW);
128)     digitalWrite(BUZZER, LOW);
129)   }
130)   else { // LOW
131)     levelText = "Low ";
```

```
132)     fullAlarm = false;
133)     setLEDs(HIGH, LOW, LOW);
134)     digitalWrite(BUZZER, LOW);
135)     setRelayAuto(true);    // ON saat low
136) }
137)
138) if (fullAlarm) handleFullBlink();
139) lcdShow();
140) }
141)
142) //
=====
143) BLYNK_WRITE(V1) {
144)     bool v = param.asInt();    // manual override,
        tetap sinkron
145)     setRelayAuto(v);
146)     lcdShow();
147) }
148)
149) //
=====
150) void setup() {
151)     Serial.begin(9600);
152)     lcd.init(); lcd.backlight();
153)
154)     pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
155)     pinMode(LED3, OUTPUT); pinMode(LED4, OUTPUT);
        pinMode(LED5, OUTPUT);
156)     pinMode(BUZZER, OUTPUT); pinMode(RELAY, OUTPUT);
157)
158)     digitalWrite(RELAY, HIGH); relayOn = false;
159)     setLEDs(LOW, LOW, LOW); digitalWrite(BUZZER,
        LOW);
160)
161)     Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
162)
163)     lcd.clear();
164)     lcd.setCursor(0,0); lcd.print("Water Level
        Init");
165)     lcd.setCursor(0,1); lcd.print("Motor: OFF");
166)     delay(1000); lcd.clear();
167)
168)     timer.setInterval(300L, ultrasonicTask);
169) }
170)
171) //
=====
172) void loop() {
173)     Blynk.run();
174)     timer.run();
175)     handleFullBlink();
176) }
```




Water Level Low, Motor ON



Water Level Medium, Motor ON



Water Level High, Motor ON



Water Level Full, Motor OFF, LED dan buzzer ON berkedip

- 2) Buat program percobaan tadi agar bisa diganti 2 mode yaitu manual seperti dalam percobaan dan otomatis seperti dalam tugas 1!

Jawab : Sebelum Memulai program tambahkan tombol switch mode MANUAL dan AUTO, dengan cara seperti pada langkah percobaan tadi, dan tambahkan program di LCD-nya untuk mengetahui sistem sedang berjalan di mode MANUAL atau AUTO.

Program :

```
1)  /* Water Level Monitoring ESP8266 + Blynk + LCD
2)    Versi: Auto/Manual Mode (V2)
3)    -----
4)    - MODE (V2): ON = AUTO, OFF = MANUAL
5)    - AUTO:
6)        LOW  -> Relay ON  (pompa hidup)
7)        FULL -> Relay OFF (pompa mati)
8)        MEDIUM/HIGH -> relay tetap
9)        Tombol V1 ikut berubah otomatis
10)   - MANUAL:
11)       V1 mengendalikan relay langsung
12)   - LCD:
13)       Baris 1: WLevel (Low/Medium/High/Full)
           (A/M)
14)       Baris 2: h:XXcm  Motor: ON/OFF
15)   - LED dan Buzzer indikator level
16)   - Relay aktif-LOW (D0)
17)  */
18)
19)  #include <LiquidCrystal_I2C.h>
20)  #define BLYNK_PRINT Serial
21)  #include <ESP8266WiFi.h>
22)  #include <BlynkSimpleEsp8266.h>
23)
24)  LiquidCrystal_I2C lcd(0x27, 16, 2);
25)
26)  // ===== Kredensial WiFi & Blynk =====
27)  char auth[] = "S4vCWAgqU_KuzDDeAFYSyO-gEUsmu6Wo";
28)  char ssid[] = "HiFi-13897";
29)  char pass[] = "psslslemanale";
30)
31)  BlynkTimer timer;
32)
33)  // ===== Pin NodeMCU =====
34)  #define TRIG    D7
35)  #define ECHO    D8
36)  #define LED3    D3    // Low
37)  #define LED4    D4    // Medium
38)  #define LED5    D5    // High
39)  #define BUZZER  D6    // Full alarm
40)  #define RELAY   D0    // Relay aktif-LOW (LOW=ON)
41)
42)  // ===== Parameter =====
43)  const int  MaxLevel  = 15;                // cm
           (maks tinggi air)
```



```

44) const float T_LOW      = MaxLevel * 0.33; // ~5 cm
45) const float T_HIGH    = MaxLevel * 0.66; // ~10 cm
46) const float T_FULL    = MaxLevel * 0.90; // ~13.5
    cm
47)
48) // ===== Variabel =====
49) bool relayOn = false;
50) bool autoMode = true;    // V2 = 1 (Auto), 0
    (Manual)
51) bool fullAlarm = false, blinkState = false;
52) unsigned long lastBlink = 0;
53) const unsigned long BLINK_MS = 300;
54)
55) int levelCm = 0;
56) String levelText = "---";
57)
58) //
    =====
59) // Fungsi pembacaan sensor ultrasonik + KALIBRASI
    (y = 0,9889 x)
60) int readDistanceCm() {
61)     digitalWrite(TRIG, LOW); delayMicroseconds(4);
62)     digitalWrite(TRIG, HIGH); delayMicroseconds(10);
63)     digitalWrite(TRIG, LOW);
64)     unsigned long t = pulseIn(ECHO, HIGH, 30000UL);
65)     if (t == 0) return MaxLevel + 200;
66)
67)     // Konversi fisika: jarak mentah (cm) = t_us /
    58.3
68)     float d_raw = t / 58.3f;
69)
70)     // Kalibrasi linear dari hasil regresi: y =
    0.9889 * x
71)     float d_cal = 0.9889f * d_raw;
72)
73)     // Kembalikan dalam cm (dibulatkan)
74)     return (int)(d_cal + 0.5f);
75) }
76)
77) //
    =====
78) // Fungsi tampilan LCD
79) void lcdShow() {
80)     const uint8_t MODE_COL = 13;    // kolom tempat
    (A)/(M)
81)     lcd.setCursor(0,0);
82)     lcd.print("WLevel ");
83)     lcd.print(levelText);
84)
85)     int used = 7 + levelText.length(); // "WLevel " =
    7
86)     while (used < MODE_COL) { lcd.print(' '); used++;
    }
87)
88)     lcd.setCursor(MODE_COL, 0);

```

```

89)   lcd.print(autoMode ? "(A)" : "(M)");
90)
91)   lcd.setCursor(0,1);
92)   lcd.print("h:");
93)   if (levelCm < 10) lcd.print('0');
94)   lcd.print(levelCm);
95)   lcd.print("cm Motor:");
96)   lcd.print(relayOn ? "ON " : "OFF");
97)   lcd.print(" ");
98) }
99)
100) //
=====
101) void setLEDs(bool L3, bool L4, bool L5) {
102)   digitalWrite(LED3, L3);
103)   digitalWrite(LED4, L4);
104)   digitalWrite(LED5, L5);
105) }
106)
107) void handleFullBlink() {
108)   if (!fullAlarm) return;
109)   if (millis() - lastBlink >= BLINK_MS) {
110)     lastBlink = millis();
111)     blinkState = !blinkState;
112)     digitalWrite(LED3, blinkState);
113)     digitalWrite(LED4, blinkState);
114)     digitalWrite(LED5, blinkState);
115)     digitalWrite(BUZZER, blinkState);
116)   }
117) }
118)
119) //
=====
120) // Fungsi kontrol relay (aktif-LOW)
121) void setRelay(bool on) {
122)   digitalWrite(RELAY, on ? LOW : HIGH);
123)   relayOn = on;
124)   Blynk.virtualWrite(V1, on ? 1 : 0); // sinkronkan
      tombol Blynk
125) }
126)
127) //
=====
128) // Logika utama (sensor + kontrol)
129) void ultrasonicTask() {
130)   int distance = readDistanceCm(); // sudah
      terkalibrasi
131)   levelCm = (distance <= MaxLevel) ? (MaxLevel -
      distance) : 0;
132)   if (levelCm < 0) levelCm = 0;
133)   if (levelCm > MaxLevel) levelCm = MaxLevel;
134)
135)   Blynk.virtualWrite(V0, levelCm); // kirim ke
      gauge
136)

```

```

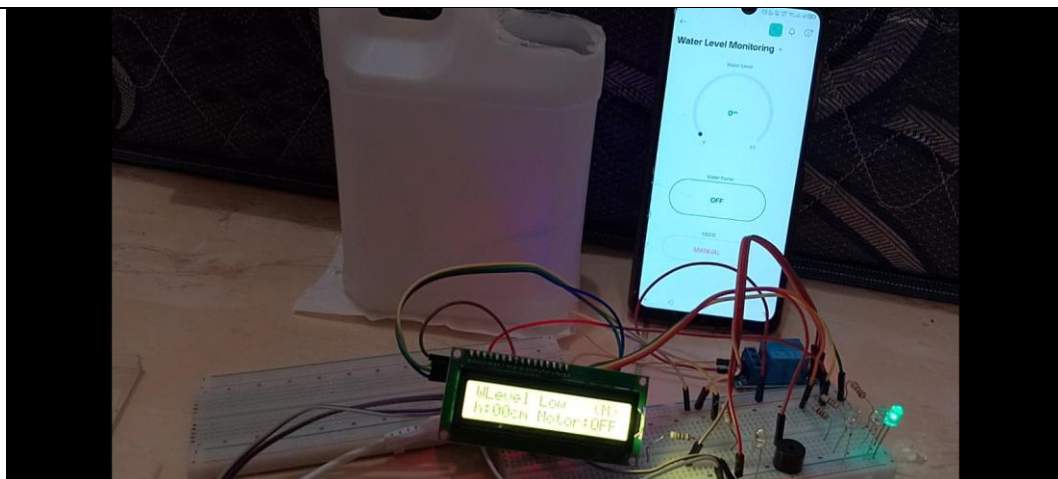
137)   if (levelCm >= (int)T_FULL) {
138)       levelText = "Full";
139)       fullAlarm = true;
140)   }
141)   else if (levelCm >= (int)T_HIGH) {
142)       levelText = "High";
143)       fullAlarm = false;
144)       setLEDs(HIGH, HIGH, HIGH);
145)       digitalWrite(BUZZER, LOW);
146)   }
147)   else if (levelCm >= (int)T_LOW) {
148)       levelText = "Medium";
149)       fullAlarm = false;
150)       setLEDs(HIGH, HIGH, LOW);
151)       digitalWrite(BUZZER, LOW);
152)   }
153)   else {
154)       levelText = "Low";
155)       fullAlarm = false;
156)       setLEDs(HIGH, LOW, LOW);
157)       digitalWrite(BUZZER, LOW);
158)   }
159)
160)   if (fullAlarm) handleFullBlink();
161)
162)   if (autoMode) {
163)       if (levelCm <= (int)T_LOW)
164)           setRelay(true);           // hidupkan pompa
165)       else if (levelCm >= (int)T_FULL)
166)           setRelay(false); // matikan pompa
167)   }
168)   lcdShow();
169)
170) //
171) // =====
172) // Handler tombol Blynk
173) BLYNK_WRITE(V1) { // Tombol Pompa Manual
174)     bool v = param.asInt();
175)     if (!autoMode) setRelay(v); // hanya di mode
176)     Manual
177)     else Blynk.virtualWrite(V1, relayOn ? 1 : 0); //
178)     abaikan di Auto
179)     lcdShow();
180) }
181) BLYNK_WRITE(V2) { // Tombol Mode Auto/Manual
182)     autoMode = param.asInt(); // 1 = Auto, 0 =
183)     Manual
184)     lcdShow();
185)     Blynk.virtualWrite(V1, relayOn ? 1 : 0); //
186)     sinkron tombol pompa
187) }
188)

```

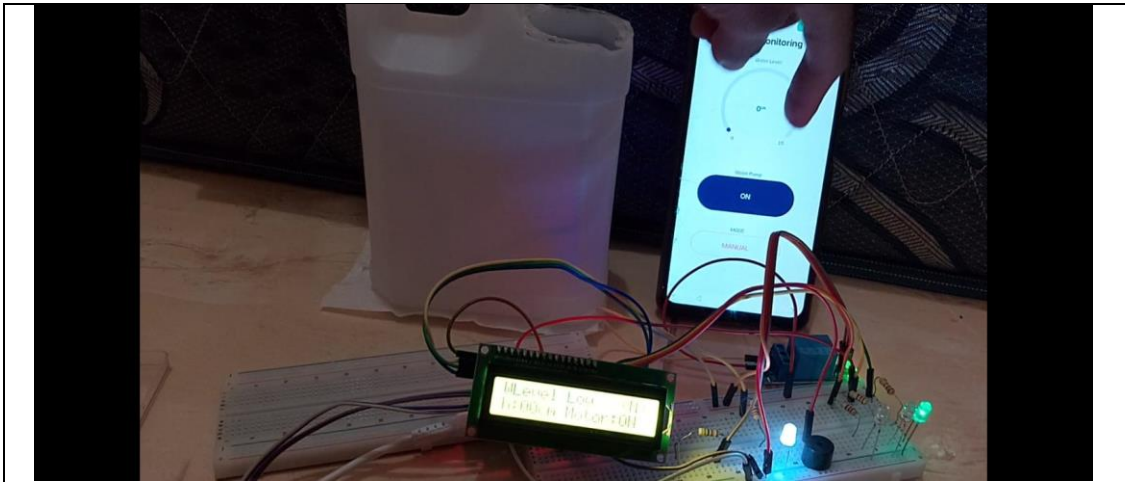
```

185) BLYNK_CONNECTED() {
186)   Blynk.syncVirtual(V1, V2);
187) }
188)
189) //
=====
190) // Setup & loop
191) void setup() {
192)   Serial.begin(9600);
193)   lcd.init(); lcd.backlight();
194)
195)   pinMode(TRIG, OUTPUT);
196)   pinMode(ECHO, INPUT);
197)   pinMode(LED3, OUTPUT);
198)   pinMode(LED4, OUTPUT);
199)   pinMode(LED5, OUTPUT);
200)   pinMode(BUZZER, OUTPUT);
201)   pinMode(RELAY, OUTPUT);
202)
203)   digitalWrite(RELAY, HIGH); relayOn = false;
204)   setLEDs(LOW, LOW, LOW); digitalWrite(BUZZER,
LOW);
205)
206)   Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
207)
208)   lcd.clear();
209)   lcd.setCursor(0,0); lcd.print("Water Level
Init");
210)   lcd.setCursor(0,1); lcd.print("Motor: OFF");
211)   delay(1000); lcd.clear();
212)
213)   timer.setInterval(300L, ultrasonicTask);
214) }
215)
216) void loop() {
217)   Blynk.run();
218)   timer.run();
219)   handleFullBlink();
220) }

```



Water Level Low, Motor OFF, Mode Manual



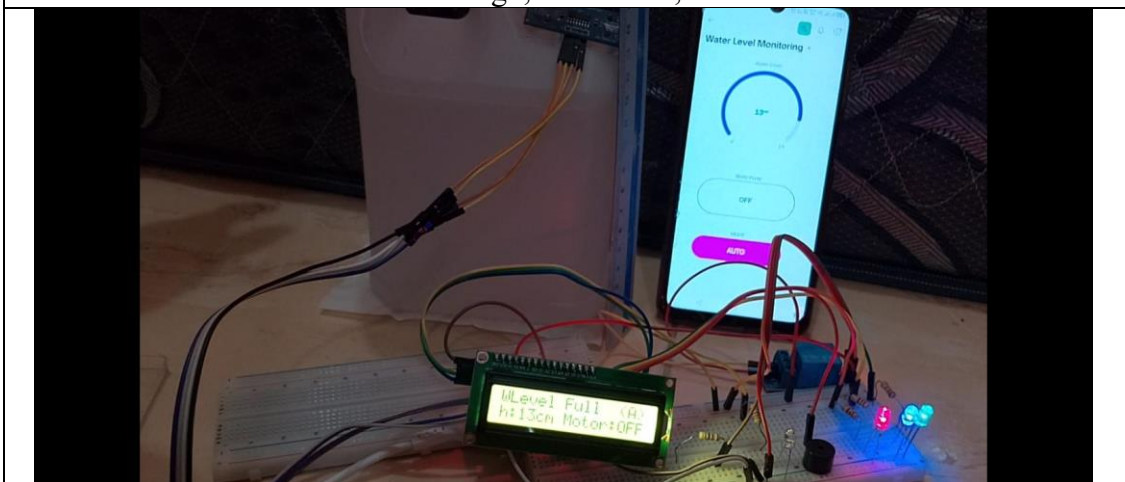
Water Level Low, Motor ON, Mode Manual



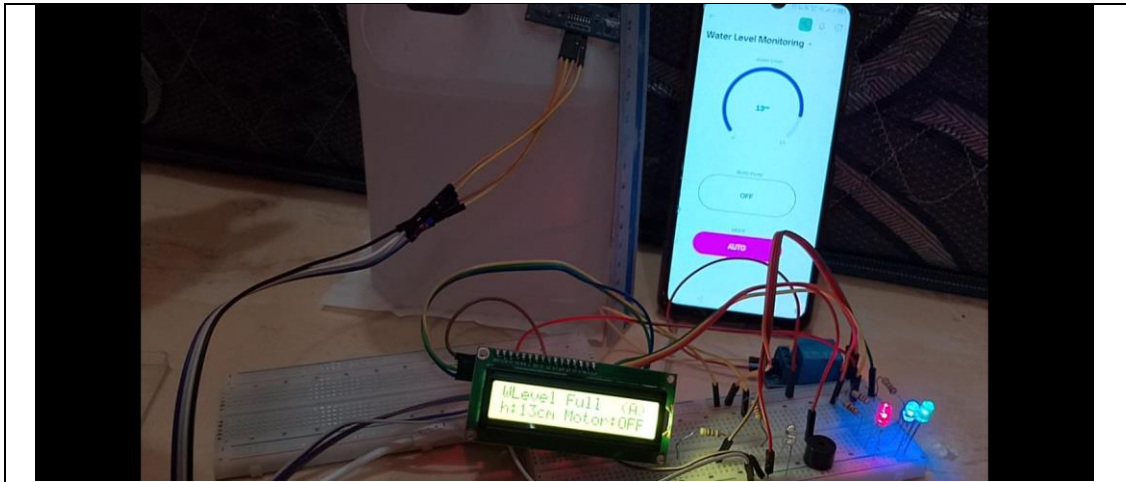
Water Level Medium, Motor ON, Mode Auto



Water Level High, Motor ON, Mode Auto



Water Level Full, Motor OFF, LED dan buzzer ON berkedip, Mode Auto



Water Level Full, Motor OFF, LED dan buzzer ON berkedip, Mode Manual



Water Level Full, Motor ON, LED dan buzzer ON berkedip, Mode Manual

VIII. Pembahasan

1. Prinsip Kerja Sensor HC-SR04

Sensor HC-SR04 bekerja dengan memancarkan gelombang ultrasonik melalui pin TRIG, lalu mendeteksi pantulan gelombang tersebut melalui pin ECHO. Waktu tempuh gelombang dikonversi menjadi jarak menggunakan rumus fisika:

$$\text{Jarak} = t_{\text{echo}} / 58,3$$

Hasil pengukuran kemudian dikalibrasi menggunakan persamaan linear hasil regresi:

$$y = 0,9889x \quad \text{atau} \quad x = 0,9889y$$

yang menunjukkan bahwa pembacaan sensor sangat mendekati jarak sebenarnya.

2. Kalibrasi dan Akurasi

Berdasarkan data pengujian, didapatkan rata-rata kesalahan pengukuran (total error) sebesar 3,6%. Nilai ini termasuk kecil, artinya sensor HC-SR04 mampu memberikan hasil yang akurat dan stabil untuk rentang pengukuran 0–15 cm. Korelasi antara jarak aktual dan hasil pembacaan sensor bersifat linear dengan $R^2 = 0,9997$, menunjukkan hubungan yang hampir sempurna antara nilai terukur dan nilai sebenarnya.

3. Logika Pengendalian

Pada mode otomatis (Auto Mode), sistem akan menghidupkan pompa saat level air berada di bawah batas LOW (~5 cm) dan mematikannya saat mencapai batas FULL (~13,5 cm). Mode Manual memungkinkan pengguna mengontrol pompa secara langsung melalui tombol V1 pada aplikasi Blynk. Indikator LED D3, D4, D5 menampilkan kondisi level air (Low, Medium, High), dan buzzer D6 akan berkedip sebagai alarm saat tangki penuh.

4. Tampilan dan Komunikasi

LCD 16x2 menampilkan informasi secara real-time berupa *Water Level*, *tinggi air (h)*, *status motor (ON/OFF)*, serta *mode kerja (A/M)*. Aplikasi Blynk IoT digunakan untuk menampilkan data level air (V0), tombol pompa (V1), dan mode Auto/Manual (V2). Seluruh komunikasi berlangsung stabil menggunakan koneksi Wi-Fi melalui modul ESP8266.

5. Analisis Kinerja Sistem

Dari hasil uji coba, sistem dapat bekerja secara real-time dan responsif, dengan pembacaan jarak yang stabil dan kesalahan minimal. Sistem mampu menjaga level air pada batas optimal tanpa perlu intervensi pengguna saat mode otomatis aktif. Adanya mode manual memberikan fleksibilitas tambahan bila dibutuhkan kontrol langsung.

IX. Kesimpulan

Hasil pengujian menunjukkan bahwa sistem *Water Level Monitoring* ini bekerja efisien, akurat, dan stabil. Dengan rata-rata error 3,6% serta fungsi Auto/Manual yang berjalan baik, alat ini dapat diandalkan sebagai prototipe sistem pengendalian level air otomatis untuk penggunaan rumah tangga maupun aplikasi edukatif di laboratorium.



LAMPIRAN VIDEO

https://drive.google.com/file/d/1d3YwxX_hWlrr8VKdUcmZmYyVtV1oBn3o/view?usp=drivesdk